
A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)

Affiliation

Address

email

1 **Revision note.** All text, tables and figures added in this revision are typeset in red (new figures
2 carry a red frame); everything in black is unchanged from the submitted version. The iteration-based
3 experiments of the submitted version on 31^2 and 63^2 grids with the LSTM router, and their appendix
4 analyses, have been removed: they are superseded by the wall-clock study of Sections E and 7,
5 which covers the same solvers and equations (plus anisotropic and variable-coefficient diffusion)
6 on 128^2 – 512^2 grids and also reports the iteration-based metrics. The Dirichlet experiment of the
7 submitted version has been withdrawn (Section D); one black sentence of the abstract (the AUC
8 sentence) was edited and is now marked in red.

9 **Status of the confirmatory re-run (draft for review).** In response to five rounds of review, every
10 wall-clock result is being regenerated under a revised, frozen protocol (Section E.1): compiled
11 classical kernels, a corrected forcing sampler, per-operation costs measured in the form in which
12 they are charged, the deployed rule equal to Algorithm 1 on macro-actions, HINTS with its two-
13 dimensional period $\tau = 15$, residual-decay controls executed live, classical baselines timed in the
14 same session as the policies with validated Krylov replays, a drift guard with re-timing, two-sided
15 tests with bootstrap intervals and explicit censoring classes, a fresh test seed and three warmed-up
16 timed replays. At the time of this draft the following are *final*: the theory corrections (Sections 4 and 5
17 and their proofs) and the wording of every claim, which is macro-driven and will be recomputed
18 from the complete tables. The 128^2 pairwise cells of Poisson, convection–diffusion and anisotropic
19 diffusion were produced by an earlier revision of the driver (before the live decay rule, the Krylov
20 validation, the stored decision sequences and the in-cell re-timing existed) and every one of their
21 instances is re-timed at the end of the chain with the final driver; until then the macros that depend
22 on those fields (decision-sequence agreement, Krylov validity, re-timing counts) print as pending.
23 Everything marked **[pending]** in the text or shown as a **[results pending]** table is still running, in
24 the order of `run_final.sh`: the 128^2 pairwise cells, the 128^2 ensembles, all 256^2 cells with their
25 ensembles, all 512^2 cells, usage traces, seed trials, overheads, the assumption, screening and theorem
26 checks and the discretisation study; then variable-coefficient diffusion at 128^2 and 256^2 (Section E.5),
27 the decision-granularity ablation (Section E.6), and the re-timing pass. Summary ranges quoted
28 in the abstract and in Section 7 are computed from the cells available at build time. Where a first
29 confirmatory run (before the protocol revision) had produced a number that is quoted in the text, it is
30 labelled as such.

Abstract

31 When solving PDEs, classical numerical solvers are often computationally ex-
32 pensive, while machine learning methods can suffer from spectral bias, failing
33 to capture high-frequency components. Designing an optimal hybrid iterative
34 solver—where, at each iteration, a solver is selected from an ensemble of solvers to
35 leverage their complementary strengths—poses a challenging combinatorial prob-
36 lem. While greedy selection is desirable for its constant-factor approximation
37 guarantee to the optimal solution under Lipschitz and shared-eigenbasis assump-
38 tions, it requires knowledge of the true error at each step, which is unavailable in
39 practice. We address this by proposing an approximate greedy router that efficiently

mimics a greedy approach to solver selection. Empirical results on the Poisson, convection–diffusion and anisotropic diffusion equations compare our method with single-solver baselines and with existing hybrid approaches such as HINTS on the area-under-the-curve (AUC) of the error trajectory: over the 128^2 cells the mean-AUC ratios of HINTS and of the solver alone to our method are $3\text{--}10822\times$ and $3\text{--}162369\times$ (one-sided paired t -tests on $\log \text{AUC}$, $p \leq 10^{-10}$). With a cost-aware instantiation of the greedy rule, a lightweight router, and a band-limited scale-equivariant corrector, we also measure wall-clock time to a target accuracy (the time charged to each method in a replay of its decisions, Section E.1) on 128^2 to 512^2 grids with compiled classical kernels, four equations (Poisson, convection–diffusion, anisotropic and variable-coefficient diffusion) and classical solvers from Jacobi to geometric multigrid, against HINTS, the best fixed schedule and the best residual-decay rule chosen per setting on the test set, and multigrid, preconditioned Krylov and direct solves timed in the same session. At truncation-level accuracy the learned router executes the same decision sequence as the best static schedule found by a test-set search (one corrector call, then smoothing) on [pending]–[pending]% of the instances, and its paired time ratios are $0.95\times\text{--}0.98\times$ against that schedule and $0.95\times\text{--}3.58\times$ against HINTS with its published period; below truncation level the ratios are $0.92\times\text{--}3.20\times$ against HINTS, $0.97\times\text{--}52.2\times$ against the one-shot schedule and $0.82\times\text{--}1.24\times$ against the best fixed schedule (the router is faster in 4 of 12 pairings), and the ratio of multigrid alone to the router is $0.03\times\text{--}0.92\times$ (all at 128^2 ; a ratio above one favours the router). Routing over ensembles of classical solvers, the larger grids and the finer decision granularity are [pending].

1 Introduction

Natural phenomena and engineered systems are often governed by ordinary and partial differential equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay, 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010], and tackling inverse problems based on external measurements (e.g., reconstructing brain activity from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which discretize the spatial and/or temporal domain and approximate the solution by solving a system of equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006, Brenner, 2008] construct a system of equations based on local basis functions, and then rely on numerical linear algebra techniques—such as the Jacobi and Gauss-Seidel method [Saad, 2003]—to compute the solution. These iterative methods, however, **must be re-run from scratch for every new** initial condition, boundary condition, or forcing function, as even minor changes to any of these parameters require re-solving the entire system of equations from scratch.

These challenges have motivated the development of neural operators [Kovachki et al., 2023]—a class of machine learning models that aim to learn the solution operator directly, enabling fast inference across a range of parameters. Neural operators lift the classical linear layer in neural networks to an operator—typically a kernel integral operator—between function spaces. Despite strong approximation guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning low-frequency components of the solution function and often struggle to capture high-frequency features that iterative solvers excel at.

Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that interleaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While HINTS reports significant empirical gains in convergence and accuracy over the standalone neural operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction may increase the error and undo recent progress.

Although state-dependent solver schedules are desirable, this naturally suggest a reinforcement learning formulation for learning routing policies. However, in our setting, the structure of iterative PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when the final error satisfies (weak) supermodularity—so that applying a beneficial solver earlier is at least as valuable as applying it later. To support this approach, we make the following contributions:

- We present a general hybrid PDE solver in which a routing rule chooses, at each iteration, from a set of classical solvers and neural operators. With oracle access to the true error at every step, we show that a greedy routing rule achieves **a constant fraction of the optimal error reduction** for linear PDEs, provided the updates are **non-expansive (Lipschitz with constant ≤ 1 , at least one of those in the optimal sequence strictly contractive)** and zero-preserving—conditions under which weak supermodularity holds—and **share a unitary eigenbasis, which gives the postfix monotonicity the guarantee also needs; for other ensembles the slack μ is measured, over short horizons only.**
- Given that the true error is not observed at test time, a convex surrogate loss is introduced, which, when minimized, enables the learned model to imitate the greedy router without access to true error information. This alignment is guaranteed through Bayes consistency **of the surrogate for the full-information router (Section 5).**
- We evaluate the approximate greedy router against HINTS and individual solvers on the Poisson, convection-diffusion, anisotropic and variable-coefficient diffusion equations, **in iterations and in wall-clock time on 128^2 to 512^2 grids with compiled classical kernels and with the classical baselines timed in the same session. At truncation-level accuracy the router executes the same decision sequence as the best static schedule (one corrector call, then smoothing) on **[pending]–[pending]**% of the instances without a search, and its paired time ratios are $0.95\times$ – $0.98\times$ against that schedule and $0.95\times$ – $3.58\times$ against HINTS with its published period; below truncation level the ratios are $0.92\times$ – $3.20\times$ against HINTS, $0.97\times$ – $52.2\times$ against the one-shot schedule and $0.82\times$ – $1.24\times$ against the best fixed schedule chosen on the test set (faster in 4 of 12 pairings at 128^2), while the ratio of multigrid alone to the router is $0.03\times$ – $0.92\times$ there. The construction extends to ensembles of classical solvers (Section E.7; **[pending]**).**

2 Background

Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \quad (1)$$

where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the coefficient function a , f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and b is the boundary term. Such PDEs can be solved numerically using various discretization strategies, such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd, 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with difference quotients (e.g., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D . For a function $g : D \rightarrow \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$ is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \quad (2)$$

with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which retains function space formulations for discretization invariance, we follow the conventions of classical numerical analysis and represent functions and operators as vectors and matrices. Discretization invariance is not essential here, since we focus on fixed-grid problems.

2.1 Iterative PDE Solvers

Direct methods like Gaussian elimination and the Thomas algorithm can be computationally expensive in high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods

like Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C \left(f_h - \mathcal{L}_h^a u^{(t)} \right), \quad (3)$$

where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$, where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

2.2 Neural Operators

Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i = \mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f) \sim \mathcal{P}} \left[\int_D \|\mathcal{G}^*(a, f)(x) - \mathcal{G}_\theta(a, f)(x)\|_2^2 dx \right]$$

Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which employ a trunk-branch decomposition to map input functions to output functions [Lu et al., 2021].

2.3 Greedy Optimization

Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \rightarrow \mathbb{R}$ is a set function defined on subsets of a set Ω . An exhaustive search is infeasible, so an efficient alternative is the greedy algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal gain, i.e. $\omega^* = \arg \max_{\omega \in \Omega \setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$; intuitively, delaying an addition cannot increase its benefit. For set function minimization problems, supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

The suboptimality of the greedy algorithm has been extensively studied for both set-function minimization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011, Bian et al., 2017, Harshaw et al., 2019] when standard assumptions—non-negativity, monotonicity, and sub/supermodularity—are weakened. In contrast, results on greedy sequence maximization [Streeter and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al., 2024, Tschitschek et al., 2017]—where the ordering of the elements affects the function—have led to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the suboptimality of the greedy solution to minimizing the final error.

3 General framework for Hybrid Solvers

To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t} \left(f_h - \mathcal{L}_h^a u_h^{(t)} \right) \quad (4)$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \dots, K\}$ indexes the function chosen at step t . Here, we use “preconditioning function” broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with

184 $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given
 185 by $S_t = \mathbf{1}_{(t+1) \bmod \tau > 0} + 1$, which selects C_1 every τ steps (first at iteration τ , after $\tau - 1$ classical
 186 steps, as in Algorithm S3 of Zhang et al. [2024]) and C_2 otherwise.

187 Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t . Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t} \left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)} \right) = (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(t)})$$

188 where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j . Here, “ $\circ$ ”
 189 denotes function composition (matrix multiplication for linear updates). The objective is to select a
 190 sequence $S = (S_1, \dots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \leq T} h(S) := \left\| (I - C_{S_{|S|}} \circ \mathcal{L}_h^a) \circ \dots \circ (I - C_{S_1} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \quad (5)$$

191 with compositions applied from right to left, so that the S_1 update acts on the initial error.

192 4 Greedy Algorithm

193 Equation (5) defines a combinatorial optimization problem with a search space exponential in
 194 T , making exact optimization intractable for large T or K . As a starting point, we consider an
 195 “omniscient” greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic
 196 since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a
 197 useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is
 198 Bayes consistent with this omniscient rule: for the full-information router, in the infinite-sample limit,
 199 it recovers the greedy decision at every visited state, which is a per-decision statement (Section 5
 200 states what the feature-based router inherits); the sequential guarantee below is stated for the rule
 201 with access to the true error.

202 As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule—such
 203 as the one described in Algorithm 1—enjoys constant-factor approximation guarantees. However,
 204 classical results focus on *set* functions, with only recent extensions made to sequences. Building on
 205 this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
 $S^0 \leftarrow \emptyset$
for $t < T$ **do**
 $S^{t+1} \leftarrow S^t \oplus \arg \min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
 $e_h^{(t+1)} \leftarrow (I - C_{S^{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
end for
return S^T

206 Let Ω^* denote the space of sequences with elements in Ω . For $S, S' \in \Omega^*$, we denote their
 207 concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function
 208 $g : \Omega^* \rightarrow \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \leq g(S)$ for all $S, S' \in \Omega^*$, and
 209 postfix monotonically non-increasing if $g(S' \oplus S) \leq g(S)$ for all $S, S' \in \Omega^*$. A prefix non-increasing
 210 function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \geq g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \quad (6)$$

211 However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we
 212 introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular
 213 with respect to $S' \in \Omega^*$ if there exists a single constant $\alpha(S') \geq 1$ such that, for every $S \in \Omega^*$,

$$g(S) - g(S \oplus S') \leq \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \quad (7)$$

214 The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence super-
 215 modularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) -$
 216 $g(S \oplus (S'_1, \dots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is
 217 controlled by the effect of appending S'_i directly to S . Thus, postponing the inclusion of S'_i cannot
 218 yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies
 219 the extent to which future gains from delays may exceed immediate gains.

220 **Postfix monotonicity enters the analysis only through the greedy prefixes, so we also allow a slack:**
 221 **g is μ -postfix monotone with respect to O along a sequence S^T if $g(S^t \oplus O) \leq \mu g(O)$ for every**
 222 **prefix S^t , $t < T$; a postfix non-increasing function satisfies this with $\mu = 1$, and μ can be computed**
 223 **exactly for short horizons (Section E.10). Having introduced these notions, we now characterize the**
 224 **suboptimality of greedy solutions of weakly supermodular and postfix monotonic sequence functions**
 225 **in Theorem 4.1.**

226 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a non-negative, prefix non-increasing function that is weakly*
 227 *supermodular with respect to the optimal solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity*
 228 *ratio of $\alpha(O)$, and let the greedy solution of length T be S^T . Suppose g is μ -postfix monotone with*
 229 *respect to O along S^T ($\mu = 1$ if g is postfix non-increasing). If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(S^T) \leq \mu (1 - \phi_T(\alpha(O))) g(O) + \phi_T(\alpha(O)) g(\emptyset), \quad \text{i.e.,} \quad g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O))) (g(\emptyset) - \mu g(O)).$$

230 The proof of Theorem 4.1 appears in Section A.2. As $T \rightarrow \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases
 231 to $1 - e^{-1/\alpha(O)}$, so, for a fixed α , the worst-case performance of the greedy rule saturates rather
 232 than degrades with horizon. Since $\alpha(O)$ and μ are defined through the optimal sequence of length
 233 T , a horizon-uniform statement needs uniform bounds $\alpha(O_T) \leq A$ and $\mu_T \leq M$, under which the
 234 guaranteed reduction is at least $(1 - e^{-1/A}) [g(\emptyset) - M g(O_T)]$ whenever that bracket is nonnegative,
 235 and only the trivial nonnegative reduction of prefix monotonicity otherwise. The bound concerns
 236 the reduction of the error over T steps relative to the best sequence of T steps: with $\alpha = \mu = 1$ it
 237 guarantees a $(1 - e^{-1})$ fraction of the optimal reduction, uniformly over ensembles, but it is not a
 238 convergence rate—a single convergent solver applied T times already reduces the error by ρ^{2T} , which
 239 for large T is far below the $e^{-1} g(\emptyset)$ term. Section E.10 reports, for short horizons, how far the greedy
 240 sequence is from the exhaustive optimum in the settings of Section E. Additionally, larger $\alpha(O)$,
 241 which indicates higher reward for delayed inclusions, loosens the suboptimality bound. Theorem 4.1
 242 requires that the sequence objective h be weakly supermodular with respect to the optimal solution
 243 and postfix monotone—properties established in Proposition 4.2. We defer the proof to Section A.3.

244 **Proposition 4.2.** *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is*
 245 *ρ_j -Lipschitz continuous with $\rho_j \leq 1$, that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$, and that $\sum_{i=1}^T (1 - \rho_{O_i}^2) > 0$,*
 246 *i.e., at least one action of the optimal sequence O is a strict contraction. Then, the function h is*
 247 *weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

248 *Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by a*
 249 *unitary matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also postfix*
 250 *non-increasing; invertibility alone does not suffice (Section A.3).*

251 The conditions of Proposition 4.2 are natural. The corrector of Section 7 leaves the modes outside its
 252 band untouched and has $\rho = 1$ exactly; the proposition permits this provided the optimal sequence
 253 contains at least one action that is a strict contraction in the norm of h (the Euclidean norm) on
 254 a subspace that contains the initial error and is invariant under every action. For the Jacobi-type
 255 smoothers the mean-free, Nyquist-free subspace is such a subspace and they are strict contractions on
 256 it; Gauss–Seidel, SOR and multigrid neither preserve it nor contract in the Euclidean norm (SOR
 257 expands, Section E.10), so for them Proposition 4.2 in the norm of h is not established. For the exact
 258 common-unitary case Proposition 4.3 applies without any strict contraction. For classical solvers, the
 259 Lipschitz constant is $\|I_N - C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs
 260 (i.e., the update is damping errors). For neural networks, Lipschitz continuity can be enforced via
 261 weight regularization [Gouk et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$
 262 well enough to make the Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both

natural and desirable: it precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical schemes, and it can be enforced for any learned model by excluding bias terms.

The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is governed by the collective contraction factors of the solvers chosen by the optimal solution: as $\sum_{i=1}^T \rho_{O_i}^2 \rightarrow T$, $\alpha(O)$ grows, resulting in a weaker bound. Postfix monotonicity ($\mu = 1$) holds when the error propagation maps share a unitary eigenbasis—constant-coefficient periodic operators with Jacobi-type smoothers and band-limited spectral correctors—but it can fail for non-commuting contractions even when all of them are invertible, and a Gauss–Seidel sweep or a learned corrector need not be invertible at all. For the non-commuting solvers of our experiments no a priori bound on μ is available; Section E.10 computes μ , $\alpha(O)$ and the bound of Theorem 4.1 by exhaustive enumeration for short horizons ($T \leq 4$ macro-actions) on every ensemble of the study, including those with Gauss–Seidel, SOR and multigrid, and reports the largest μ ([pending]), the largest $\alpha(O)$ over greedy prefixes ([pending]) and the number of instance–ensemble pairs on which the theorem’s premises were verified ([pending]); at the horizons of tens to hundreds of macro-actions at which the oracle actually runs, neither α in the Euclidean norm nor μ is established for the non-normal sweeps, and no guarantee is claimed for them.

Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis, supermodularity is established. This occurs, for example, for linear, constant-coefficient PDEs with periodic boundary conditions when the solver ensemble consists of (damped) Jacobi iterations and band-limited spectral correctors such as a single-layer linear Fourier neural operator, all of which are diagonal in the Fourier basis; lexicographic Gauss–Seidel and SOR are not, because their sweep order breaks translation invariance. The proof of Theorem 4.3 is deferred to Section A.5.

Proposition 4.3. *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common unitary matrix P (orthogonal in the real symmetric case; the Jacobi matrix of the convection–diffusion operator is a real circulant with complex eigenvalues, hence unitarily but not orthogonally diagonalisable). Then, h is supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware by changing its action set, not its comparison: let c_j denote the (measured) cost of one application of C_j , take the cost u of one neural-operator call as the unit, and replace each preconditioner by the macro-action $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \text{round}(u/c_j)\}$ consecutive iterations, so that every action not dearer than the unit costs u up to rounding ($m_j c_j / u$ within $1/(2m_j - 1)$ of 1). For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m -fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is ρ_j -Lipschitz and zero-preserving, then so is its m_j -fold composition (with constant $\rho_j^{m_j}$), a shared eigenbasis is inherited by powers, and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to an ensemble of macro-actions, all of which are equalised to the unit cost. The deployed rule is exactly Algorithm 1 on these macro-actions—the plain comparison of the errors after one macro-action—and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$; it selects, at every decision epoch, the action with the largest error reduction per unit cost up to the rounding of the m_j . The one exception is an action dearer than the unit ($c_j > u$, hence $m_j = 1$), whose error reduction is compared per unit of cost, $(\|e_j\|/\|e^{(t)}\|)^{u/c_j}$: this is a geometric-rate heuristic rather than an equal-cost step, and the analysis does not cover it. Which actions are dearer than the unit is a fact of the measured cost tables (Tables 7, 13 and 19): on the 128^2 grids measured so far (Poisson, convection–diffusion, anisotropic diffusion) none is, so those cells run Algorithm 1 on macro-actions, which is also the rule that the exhaustive check of Section E.10 verifies (variable-coefficient diffusion [pending]); on the larger grids only a multigrid cycle can be dearer than the unit ([pending]), and in the granularity ablation of Section E.6 the corrector itself is. Even with all exponents equal to one, the wall-clock reading of a horizon of T macro-actions is approximate: the rounding of the m_j and the router’s decision cost are not part of the equal-count analysis. (An earlier version applied the exponent $u/(m_j c_j)$ to

every action, including the rounding error of the cheap ones; that rule is not Algorithm 1, and it was changed so that the theory covers what is deployed.) Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t , router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j , resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) = \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \quad (8)$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} [l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router's own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t-1)}) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*}(f_h - \mathcal{L}_h^a u_h^{(t-1)})$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h) . At each t , l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$. Thus the input distribution seen by r depends on (a_h, f_h) , not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 \quad (9)$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

5.1 Surrogate Loss

Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice. We instead introduce a surrogate loss that (1) is convex in the scores; (2) upper bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes consistent with respect to the loss l if

$$\lim_{n \rightarrow \infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* = 0 \implies \lim_{n \rightarrow \infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^* = 0$$

where $\mathcal{R}_l(f) = \mathbb{E}[l(f(X), Y)]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data, if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ , it also converges to the optimal risk with respect to the original loss l .

To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$ with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \rightarrow \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$. For

example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss yields the same decision as minimizing Equation (8):

$$\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) = - \sum_{j=1}^K \sum_{k=1}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h) \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{m=1}^K \exp(g_m(a_h, f_h, u_h^{(t)}))} \right) \quad (10)$$

where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ -risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to the score vector $\mathbf{g}$ follows from the convexity of the negative log-softmax function in its inputs (the surrogate is convex in the scores, not in the parameters of a network that produces them). Moreover, Ψ upper bounds l_{route} up to a constant factor, and we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that the per-state normalised surrogate $\hat{\Psi}$ is Bayes consistent with respect to the normalised routing loss $\hat{l}_{\text{route}}$, which has the same Bayes decision as l_{route} at every state; the proof can be found in Section B.3.

Theorem 5.1. *Let $K \geq 2$ and let the per-step costs be normalised state by state, $\hat{c}_j = \tilde{c}_j / \sum_{k=1}^K \tilde{c}_k$, at every state at which $\sum_k \tilde{c}_k > 0$, i.e., at which at least one solver does not annihilate the error (states at which every solver annihilates it carry no routing loss and are discarded), and let $\hat{l}_{\text{route}}$ and $\hat{\Psi}$ denote the routing loss and the surrogate with $\tilde{c}_j$ replaced by $\hat{c}_j$; the normalisation does not change the Bayes-optimal decision equation 9 at any state. Then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, $\hat{\Psi}$ is a Bayes consistent surrogate for $\hat{l}_{\text{route}}$.*

Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy with instance-dependent weights $\sum_{k \neq j}^K \hat{c}_k(a_h, u_h^{(t)}, u_h)$. The normalisation is what the router is trained with (Section E.1) and what makes the calibration constants meaningful: an earlier version assumed uniform absolute bounds $E_{\min} \leq \max_j \tilde{c}_j \leq \bar{E}$, which cannot hold along a convergent solve, where every $\tilde{c}_j \rightarrow 0$; with the normalised weights the only hypothesis is that at least one solver leaves a positive fraction of the error at every visited state, which Section E.10 checks. Under this condition, minimizing $\hat{\Psi}$ recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). The proof bounds the conditional routing excess by the square root of the conditional surrogate excess through Pinsker’s inequality, a direct calibration argument in the spirit of Mao et al. [2023], and takes expectations; it concerns unrestricted measurable scores, not a fixed network class or optimiser. This is a per-decision statement on the state distribution the router is trained on: it says that minimising Ψ recovers the greedy decision at each visited state, not that the induced sequence, its time to tolerance, or its distance to the optimal sequence are controlled; the latter would require a sequential argument about the router’s own state distribution, which we do not provide and mitigate empirically with DAgger (Section E). The deployed router, moreover, does not see $(a_h, f_h, u_h^{(t)})$ but a few scalar features of the residual history (Section E.1); with respect to those features the Bayes-optimal router minimises the conditional expectation of the normalised costs given the features, which can differ from the full-information greedy decision when states with different error spectra share the same features, and the normalisation preserves the decision state by state but not necessarily after that averaging. Theorem 5.1 therefore concerns the full-information router, and the price of the feature compression is measured rather than proved, by the agreement between the deployed router and the oracle (Section E).

6 Related Works

Hybrid PDE Solvers: Early data-driven solvers sought convergence guarantees by predicting parameters—e.g., preconditioning matrices, multi-grid smoothers or restriction matrices—within iterative schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025, Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates’ ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi

steps, then one DeepONet correction), achieving faster convergence than the standalone numerical solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin, 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics—e.g., thresholding a cheap model’s uncertainty estimates [Chuang et al., 2024, 2025]—often poorly balance the cost-accuracy tradeoff, motivating many systems to instead learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: “experts” (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We evaluate the approximate greedy router on four PDEs posed on the unit domain $D = [0, 1]^2$ with periodic boundary conditions:

Poisson: $-\Delta u = f$, **ConvDiff:** $-\Delta u + 20 \partial_{x_1} u + 20 \partial_{x_2} u = f$, **AnisoDiff:** $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$, $\epsilon = 10^{-2}$, **VarCoef**

where f is centred to satisfy the compatibility condition $\int_D f(\mathbf{x}) d\mathbf{x} = 0$ and is drawn from a zero-mean hierarchical Gaussian random field with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$ and sample-dependent (α, β, γ) (Section C). Anisotropic diffusion is the standard problem on which point relaxation fails to smooth: an error mode oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep. Variable-coefficient diffusion uses a fixed smooth coefficient field $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$ with a prescribed contrast $\max a / \min a$ of 10: its discrete operator is not diagonalised by the FFT, so no direct spectral solve exists, the corrector’s band-limited inverse has no closed form and is fitted, and the fitted corrector is only approximately accurate (Section E.5). The domain is discretised on 128^2 , 256^2 and 512^2 grids; Dirichlet boundary conditions are not covered by this study (Section D). Because iteration counts favour methods whose iterations are expensive, our primary metric is the *wall-clock time to a target relative error* charged to each method in a replay of its decisions (Section E.1) $\varepsilon = \|e_h^{(t)}\| / \|u_h\|$ on 64, 32 and 16 test instances per grid, with $\varepsilon = h^2$ —a grid-dependent tolerance chosen to lie below the discretisation error of the test instances (the smallest ratio of the estimated discretisation error to h^2 over the confirmatory instances is [pending], Section E.1)—highlighted; paired comparisons of times use two-sided Wilcoxon signed-rank tests with a Holm correction and bootstrap intervals, the AUC comparisons one-sided paired t -tests on log AUC. We also report the iteration-based metric of the original submission, the area under the error curve, $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\| / \|u_h\|$ over $T = 300$ iterations. Section E gives the full protocol and collects every result per equation.

Setup. The classical solvers are Jacobi, damped Jacobi ($\omega = 0.67$), Gauss–Seidel (GS), symmetric GS (SymGS), SOR ($\omega = 1.5$), line Gauss–Seidel (anisotropic diffusion only: every grid line along the strongly coupled axis solved exactly, the classical remedy for anisotropy) and geometric multigrid (V(2,2)-cycles with GS smoothing; line-GS smoothing for anisotropic diffusion), all implemented as

compiled single-threaded C stencil kernels (a GS sweep costs $61 \mu\text{s}$ and a V-cycle $326 \mu\text{s}$ at 128^2 , measured in the form in which the benchmark charges them), so the classical baselines are not slowed by the implementation. The neural operator is a DeepONet corrector that acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly: its deployed form is a linear, Fourier-structured map (a least-squares fit of the band-limited inverse of the discrete operator, which it reproduces to $\approx 10^{-6}$ on the constant-coefficient equations), so one call solves the smooth band of the spectrum at the cost of about 32 Jacobi or 11 GS sweeps at 128^2 (and [pending] Jacobi sweeps at 512^2 , since its cost is nearly grid-independent) and never injects error outside its band. The router is made *cost-aware* through the cost-equalised macro-actions of Section 4 and selects from $6+K$ scalar features (log residual, its recent decay rate, the previous action and call counts) with a two-layer MLP whose decision costs $16\text{--}20 \mu\text{s}$ in situ, independently of N ; it is trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle on oracle rollouts followed by DAGger rounds. We compare against the classical solver alone; HINTS with the period $\tau=15$ that Zhang et al. [2024] use in all of their two-dimensional experiments ($\tau=25$, their one-dimensional period, is reported in Section E); the best fixed schedule chosen *a posteriori* on the test set for each cell from HINTS with $\tau \in \{2, 5, 10, 15, 25, 50\}$, phase-shifted HINTS (first call at iteration 0) and the static one-shot schedule (corrector once at iteration 0, then the solver); the best of three residual-decay rules—deterministic controls at the router’s granularity that call the corrector again whenever the residual’s per-iteration contraction over the last macro-action was worse (closer to one) than a threshold, also chosen on the test set; the cost-aware greedy oracle (Algorithm 1 with the true error; not deployable); and the classical methods a practitioner would use on these problems without a corrector—multigrid alone (line-smoothed for anisotropic diffusion), multigrid-preconditioned CG or BiCGSTAB, and the FFT or sparse direct solve—timed inside the same replay loop as the policies. The forcings are sampled from the hierarchical GRF with the Hermitian symmetry of a real field and without Nyquist modes (Section E.1). All configurations were fixed on a development test set, and every number reported is a single confirmatory evaluation on a fresh test set with three timed replays in random order and a drift guard per instance (Section E.9); paired comparisons use two-sided Wilcoxon tests with a Holm correction and bootstrap intervals.

Paired solvers. Table 1 reports, for every pairing $\{\text{NO}, \text{solver}\}$, equation and grid, the median wall-clock time to $\varepsilon = h^2$ of HINTS, of the best fixed schedule, and of the learned router, together with the classical methods without corrector. Every ratio below is a paired median time ratio (baseline over router; above one favours the router), computed over the 128^2 pairwise cells unless a grid is named. Two regimes appear. *At truncation level* the router makes exactly one corrector call on 80–100% of the instances (Table 6), executes the same decision sequence to h^2 as the cost-aware oracle on [pending]–[pending]% of the instances and as the best static schedule (the corrector at iteration 0, then smoothing) on [pending]–[pending]%, and its ratios against the best fixed schedule found by a test-set search are $0.95 \times \text{--} 0.98 \times$ at 128^2 ([pending]–[pending] at 256^2 , [pending]–[pending] at 512^2), a difference consistent with its decision overhead (Section E.8); against HINTS with its published period $\tau=15$, whose first call comes $\tau - 1$ sweeps too late, they are $0.95 \times \text{--} 3.58 \times$ ([pending]–[pending] at 256^2 , [pending]–[pending] at 512^2 ; $0.96 \times \text{--} 4.73 \times$ against $\tau=25$); against the best residual-decay rule, which also calls the corrector first, $0.95 \times \text{--} 1.00 \times$; and against the classical solver alone $0.95 \times \text{--} 664 \times$ (censored runs entered at the cap; the low end is the multigrid pairing, where multigrid alone reaches h^2 in a few cycles; anisotropic diffusion [pending]). *Below truncation level* the schedule matters and the picture is pairing-dependent: down to $\varepsilon = 10^{-8}$ the ratios are $0.92 \times \text{--} 3.20 \times$ against HINTS ($\tau=15$); the router is faster in 10 of 12 pairings) and $0.97 \times \text{--} 52.2 \times$ against the one-shot schedule (faster in 10), because the router calls the corrector again when the smooth error has re-emerged (Figure 1), but $0.82 \times \text{--} 1.24 \times$ against the best fixed schedule (faster in 4) and $0.97 \times \text{--} 2.02 \times$ against the best residual-decay rule (faster in 10). The best fixed schedules at 10^{-8} are $\tau=15$ (first call at 0) (3), one-shot (2), $\tau=10$ (first call at 0) (2), $\tau=5$ (first call at 0) (2), $\tau=25$ (first call at 0) (2), $\tau=10$ (1) (number of pairings in parentheses): a periodic schedule with a short period calls the corrector more often than a cost-equalised macro-action of 11 Gauss–Seidel or 32 Jacobi sweeps allows the router to, and Section E.6 tests whether that granularity explains the losses [pending]. The ratio of the router’s median time to the cost-aware oracle’s is at most $1.09 \times$, and its decision sequence to 10^{-8} is identical to the oracle’s on [pending]–[pending]% of the instances; the stability of its decisions over five independent training seeds is reported in Section E [pending]. Against the classical methods without corrector, the ratio of multigrid alone to the router is $0.95 \times \text{--} 1.09 \times$ at h^2 and $0.03 \times \text{--} 0.92 \times$ at 10^{-8} , that of multigrid-preconditioned CG or BiCGSTAB $0.95 \times \text{--} 1.64 \times$ and $0.03 \times \text{--} 0.95 \times$, and routing over $\{\text{NO}, \text{multigrid}\}$ has ratios $0.94 \times \text{--} 0.99 \times$ and

Table 1: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ for every solver pairing $\{\text{NO, solver}\}$, equation and grid ($64, 32$ and 16 test instances at $128^2, 256^2$ and 512^2): HINTS with its published two-dimensional period $\tau=15$, the best fixed schedule chosen per cell on the test set itself from HINTS ($\tau \in \{2, 5, 10, 15, 25, 50\}$), phase-shifted HINTS ($\tau \in \{5, 10, 15, 25, 50\}$, first call at 0) and the one-shot schedule (subscript), and the learned router (paired median speedup over HINTS-15/over the best fixed schedule in parentheses; bold: both ratios at least one). The paired ratio of the classical solver alone to the router is $0.95 \times -664 \times$ at 128^2 (above one favours the router; censored runs entered at the cap; the multigrid pairing is the only one near or below one; Section E). The last rows of each block give geometric multigrid alone (point-smoothed; line-smoothed as well for anisotropic diffusion), the best multigrid-preconditioned Krylov method and the direct solve, without corrector, timed in the same session on the same instances. Anisotropic diffusion is not run at 512^2 : its corrector keeps full resolution along x_1 , and the resulting $65,536 \times 65,536$ branch matrix does not fit in memory (Section E). $\dagger n$: n instances did not reach h^2 within the iteration cap.

Equation	Pairing	128 × 128			256 × 256			512 × 512		
		HINTS-15	best schedule	router (ours)	HINTS-15	best schedule	router (ours)	HINTS-15	best schedule	router (ours)
Poisson	Jacobi	1.02 ms	718 $\mu\text{s}_{\tau=15, t_0=0}$	758 μs (1.37×/0.96×)	—	—	—	—	—	—
	Jacobi (0.67)	981 μs	711 $\mu\text{s}_{\tau=5, t_0=0}$	743 μs (1.32×/0.98×)	—	—	—	—	—	—
	GS	1.58 ms	711 $\mu\text{s}_{\tau=15, t_0=0}$	752 μs (2.06×/0.95×)	—	—	—	—	—	—
	SymGS	1.99 ms	726 $\mu\text{s}_{\text{one-shot}}$	772 μs (2.53×/0.95×)	—	—	—	—	—	—
	SOR (1.5)	3.03 ms	711 $\mu\text{s}_{\tau=25, t_0=0}$	764 μs (3.58×/0.96×)	—	—	—	—	—	—
	Multigrid	933 μs	821 $\mu\text{s}_{\tau=25, t_0=0}$	874 μs (0.99×/0.97×)	—	—	—	—	—	—
	Multigrid alone (no corrector)			1.01 ms	—	—	—	—	—	—
	PCG (MG) (no corrector)			978 μs	—	—	—	—	—	—
	FFT direct solve (exact)			246 μs	—	—	—	—	—	—
ConvDiff	Jacobi	1.08 ms	781 $\mu\text{s}_{\text{one-shot}}$	833 μs (1.37×/0.95×)	—	—	—	—	—	—
	Jacobi (0.67)	1.09 ms	801 $\mu\text{s}_{\text{one-shot}}$	864 μs (1.28×/0.96×)	—	—	—	—	—	—
	GS	1.94 ms	799 $\mu\text{s}_{\tau=50, t_0=0}$	885 μs (2.20×/0.96×)	—	—	—	—	—	—
	SymGS	2.11 ms	789 $\mu\text{s}_{\tau=10, t_0=0}$	831 μs (2.50×/0.96×)	—	—	—	—	—	—
	SOR (1.5)	3.13 ms	815 $\mu\text{s}_{\tau=15, t_0=0}$	888 μs (3.52×/0.96×)	—	—	—	—	—	—
	Multigrid	1.02 ms	981 $\mu\text{s}_{\tau=15, t_0=0}$	1.04 ms (0.95×/0.96×)	—	—	—	—	—	—
	Multigrid alone (no corrector)			1.02 ms	—	—	—	—	—	—
	BiCGSTAB (MG) (no corrector)			1.47 ms	—	—	—	—	—	—
	FFT direct solve (exact)			250 μs	—	—	—	—	—	—

1.08×–1.14× against multigrid alone; on the constant-coefficient equations the router takes 2.98×–3.34× times as long as the FFT direct solve, and on variable-coefficient diffusion, which has no such solve, it is compared with the sparse direct solve (Section E.5). For the iteration-based metric of the original submission, at 128^2 the mean AUC over $T = 300$ iterations of HINTS and of the solver alone are 3–10822× and 3–162369× the router’s (one-sided paired t -tests on log AUC, $p \leq 10^{-10}$; Tables 8, 14, 20 and 26).

Solver ensembles. The same construction routes over ensembles $\text{NO} \cup \mathcal{W}$ of several classical solvers. Since the greedy rule is a per-decision comparison, a larger ensemble can only help where the error passes through regimes that favour *different* members; an oracle-level screening of every subset of up to three members (Section E.7) showed, in the development runs, that this never happens at $\varepsilon = h^2$ —one corrector call and one macro-action of the cheapest smoother already reach truncation level—and happens at 10^{-8} only through a nonstationary Jacobi schedule (undamped Jacobi on the band-edge modes, damped Jacobi on the near-Nyquist modes). We evaluate the learned router over the nested sets $\{\text{Jacobi}\}$, $\{\text{Jacobi (0.67)}\}$, $\{\text{Jacobi, Jacobi (0.67)}\}$ and $\{\text{Jacobi, Jacobi (0.67), GS}\}$ and the four larger sets of Table 30 on every equation, with the single-member routers evaluated in the same session on the same instances. The revised-protocol ensemble cells are **[pending]**. In the first confirmatory run (revision 1, 39-sweep Jacobi macro-actions) no ensemble was significantly faster than the best single-member router at 128^2 and the ensemble oracle was within 6% of the best single-member oracle, which we attributed to the decision granularity: a single damped-Jacobi macro-action of 32 sweeps contracts the band-edge modes by $0.665^{32} \approx 2 \times 10^{-6}$, which suggests that the band the corrector leaves at 128^2 is removed within one or two macro-actions, leaving a nonstationary schedule little to exploit. Whether this holds under the revised protocol, on the 256^2 grids (twice as wide a band) and at a finer granularity (Section E.6) is what the pending cells decide; the ensemble routers’ ratios against the best single-member router at h^2 and 10^{-8} will be reported as **[pending]**–**[pending]** and **[pending]**–**[pending]**. Finally, Section E.10 checks numerically which assumptions of Sections 4 and 5 hold in which setting: exactly, up to the corrector’s fit error, for the Jacobi-type ensembles on the constant-coefficient equations, and only as finite diagnostics for the non-normal sweeps and multigrid, for which no long-horizon guarantee is claimed.

Every result per equation and grid, the ensemble study, the accounting of overheads and training cost, and the verification of the theoretical assumptions are collected in Section E; Section D reports the Dirichlet experiment of the original submission.

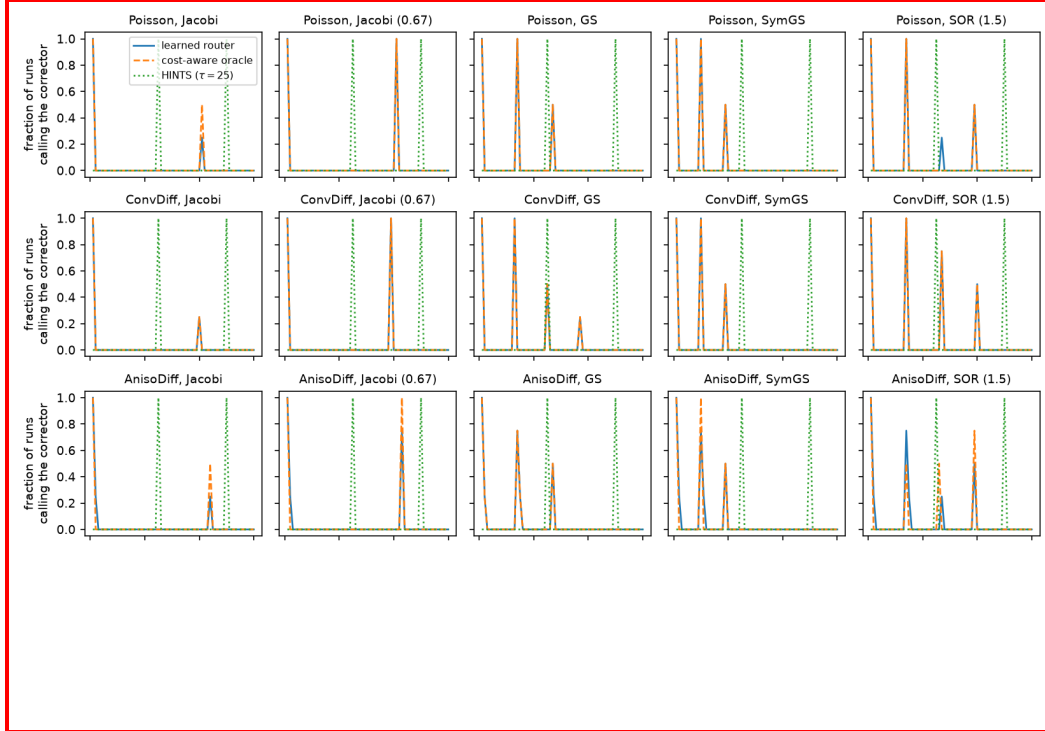


Figure 1: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=15$), for every solver pairing (columns) and all three equations (rows); the same figure for the 256^2 and 512^2 grids is in Section E.

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is trained without accounting for its downstream role in correction term prediction; jointly training the ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—executes the decision sequence of the best static schedule at truncation level on [pending]–[pending]% of the instances and gives pairing-dependent results below it (Section 7), and RL could extend it to non-myopic budgeted routing. RL would additionally enable extending the ensemble routing to *continuous* parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

- Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.
- Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a review. *Monthly weather review*, 119(9):2206–2223, 1991.
- John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.

562 Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*,
563 volume 112. American Mathematical Soc., 2010.

564 Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal*
565 *processing magazine*, 18(6):14–30, 2001.

566 Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis
567 Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the
568 inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33,
569 2008.

570 Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*.
571 Oxford university press, 1985.

572 Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-*
573 *state and time-dependent problems*. SIAM, 2007.

574 Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*.
575 Courier Corporation, 2003.

576 Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.

577 Susanne C Brenner. *The mathematical theory of finite element methods*. Springer, 2008.

578 Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.

579 Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew
580 Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with
581 applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

582 Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks
583 with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on*
584 *neural networks*, 6(4):911–917, 1995.

585 Lizuo Liu and Wei Cai. Multiscale deepnet for nonlinear operators in oscillatory function spaces for
586 building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.

587 Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator
588 learning. *Journal of Computational Physics*, 506:112944, 2024.

589 Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for
590 oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.

591 Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating
592 spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint*
593 *arXiv:2503.13695*, 2025.

594 Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of
595 deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

596 Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua
597 Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference*
598 *on machine learning*, pages 5301–5310. PMLR, 2019.

599 Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle:
600 Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

601 Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for
602 general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

603 Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and
604 George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical
605 solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

606 John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

607 Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods:*
608 *Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

609 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew
610 Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations.
611 *arXiv preprint arXiv:2010.08895*, 2020a.

612 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik
613 Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial
614 differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

615 Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning
616 nonlinear operators via deepnet based on the universal approximation theorem of operators.
617 *Nature machine intelligence*, 3(3):218–229, 2021.

618 George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for
619 maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

620 Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions.
621 In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*
622 *(APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

623 Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular
624 minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

625 Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection,
626 sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3):
627 1–34, 2018.

628 Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions.
629 *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

630 Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschiatschek. Guarantees for
631 greedy maximization of non-submodular functions with applications. In *International conference*
632 *on machine learning*, pages 498–507. PMLR, 2017.

633 Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond
634 non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on*
635 *Machine Learning*, pages 2634–2643. PMLR, 2019.

636 Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions.
637 *Advances in Neural Information Processing Systems*, 21, 2008.

638 Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions
639 and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

640 Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions
641 with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

642 Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity.
643 In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30,
644 pages 38–47, 2020.

645 Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for
646 the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint*
647 *arXiv:2409.05020*, 2024.

648 Sebastian Tschiatschek, Adish Singla, and Andreas Krause. Selecting sequences of items via
649 submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
650 volume 31, 2017.

651 Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks
652 by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

653 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
654 neural networks. *Neural computation*, 1(2):270–280, 1989.

655 Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C
656 Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks.
657 *Advances in neural information processing systems*, 29, 2016.

658 Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence
659 prediction with recurrent neural networks. *Advances in neural information processing systems*, 28,
660 2015.

661 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and
662 applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.

663 Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic
664 multigrid coarsening using reinforcement learning. *Advances in neural information processing
665 systems*, 34:12129–12140, 2021.

666 Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid
667 methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*,
668 167:217–231, 2024.

669 Alena Kopaničáková and George Em Karniadakis. Deepnet based preconditioning strategies for
670 solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1):
671 C151–C181, 2025.

672 Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks.
673 *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

674 Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters.
675 *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

676 Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay
677 Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential
678 equations. *Computational Mechanics*, 72(3):471–484, 2023.

679 Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product.
680 *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

681 Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical
682 examples. *Mathematics of Computation*, 2025.

683 Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic
684 systems. *Mathematics*, 10(21):4014, 2022.

685 Tal Shnitzer, Anthony Ou, Mírian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson,
686 and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint
687 arXiv:2309.15789*, 2023.

688 Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt
689 Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing
690 system. *arXiv preprint arXiv:2403.12031*, 2024.

691 Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS
692 Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query
693 routing. *arXiv preprint arXiv:2404.14618*, 2024.

694 Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and
695 Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level
696 scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

697 Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki,
698 Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint
699 arXiv:2410.13284*, 2024.

- 700 Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui,
701 Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based
702 on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*,
703 2025.
- 704 Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to
705 large language models. *arXiv preprint arXiv:2308.11601*, 2023.
- 706 Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to
707 large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.
- 708 Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language
709 model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on*
710 *Web Search and Data Mining*, pages 606–615, 2024.
- 711 Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of
712 local experts. *Neural computation*, 3(1):79–87, 1991.
- 713 Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural*
714 *computation*, 6(2):181–214, 1994.
- 715 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and
716 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv*
717 *preprint arXiv:1701.06538*, 2017.
- 718 Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang,
719 Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional
720 computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- 721 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
722 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
723 2022.
- 724 Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris
725 Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al.
726 Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- 727 Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In
728 *Backpropagation*, pages 137–169. Psychology Press, 2013.
- 729 Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*,
730 volume 11. University of Cambridge Department of Engineering Cambridge, 1987.
- 731 Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model.
732 *Neural networks*, 1(4):339–356, 1988.
- 733 Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and
734 George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with prac-
735 tical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*,
736 393:114778, 2022.
- 737 Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured
738 prediction to no-regret online learning. In *Proceedings of the Fourteenth International Confer-*
739 *ence on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference
740 Proceedings, 2011.

741 A Proofs for Section 4

742 A.1 Proof of Proposition A.1

743 **Proposition A.1.** Any prefix monotonically non-increasing sequence supermodular function g is
744 weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$

745 *Proof.* This proof is adapted from Liberty and Sviridenko [2017].

746 If g is sequence supermodular then,

$$\begin{aligned}
& g(S) - g(S \oplus S') \\
&= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) - g(S \oplus (S'_1, \dots, S'_i)) \\
&\stackrel{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\
&\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i)
\end{aligned}$$

747 (a) by supermodularity

748 □

749 A.2 Proof of Theorem 4.1

750 **Theorem 4.1.** Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal
751 solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity ratio of $\alpha(O)$ and μ -postfix monotonicity
752 with respect to O along the greedy solution S^T of length T . If $\phi_T(\alpha) = \left(1 - \frac{1}{\alpha T}\right)^T$, then

$$g(S^T) \leq \mu(1 - \phi_T(\alpha(O)))g(O) + \phi_T(\alpha(O))g(\emptyset)$$

753 *Proof.* This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko
754 [2017]

$$\begin{aligned}
g(S^t) - \mu g(O) &\stackrel{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\
&= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \dots, o_{i-1})) - g(S^t \oplus (o_1, \dots, o_i)) \\
&\stackrel{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)|O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)Tg(S^t) - \alpha(O)T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\
&= \alpha(O)Tg(S^t) - \alpha(O)Tg(S^{t+1})
\end{aligned}$$

755 (a) by μ -postfix monotonicity along S^T , i.e., $g(S^t \oplus O) \leq \mu g(O)$; (b) by weak supermodularity with
756 respect to O (applied to the prefix S^t).

757 After rearranging the inequality, we get:

$$\begin{aligned}
g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} (\mu g(O) + (\alpha(O)T - 1)g(S^t)) \\
&= \frac{\mu}{\alpha(O)T} g(O) + \left(1 - \frac{1}{\alpha(O)T}\right) g(S^t)
\end{aligned}$$

758 When recursively applying this inequality, we get:

$$\begin{aligned}
g(S^T) &\leq \frac{\mu}{\alpha(O)T} g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\
&= \mu \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right) g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset)
\end{aligned}$$

759

□

760 A.3 Proof of Proposition 4.2

761 **Proposition 4.2.** Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is
762 ρ_j -Lipschitz continuous with $\rho_j \leq 1$, that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$, and that $\sum_{i=1}^T (1 - \rho_{O_i}^2) > 0$,
763 i.e., at least one action of the optimal sequence O is a strict contraction. Then, the function h is
764 weakly supermodular with respect to the optimal solution O , with

$$\alpha(O) = \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

765 Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by a
766 unitary matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also postfix
767 non-increasing; invertibility alone does not suffice (Section A.3).

768 *Proof.* For brevity, we use the notation $(g_1 \circ \dots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is
769 applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz
770 continuous functions:

- 771 • **Property 1:** If g is ρ -Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 =$
772 $\|g(x) - g(0)\|_2 \leq \rho \|x - 0\|_2 = \rho \|x\|_2$
- 773 • **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
774 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is **at most** $\rho_1 + \rho_2$.
- 775 • **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
776 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is **at most** $\rho_1 \rho_2$.

777 In order to prove weakly- α -supermodularity, we must first prove prefix monotonicity.

778 **Prefix monotonicity:** Let $S \preceq S'$ where $S' = S \oplus N$.

$$\begin{aligned}
h(S') &= \left\| \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&= \left\| \circ_{t=|S'|}^{|S|+1} (I - C_{S'_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&\stackrel{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\
&\leq h(S)
\end{aligned}$$

779 (a) by Property 1

780 **Weak supermodularity:** We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) - h(S \oplus$
781 $O)$ and a lower bound for $\sum_{i=1}^T h(S) - h(S \oplus O_i)$. Since $h \geq 0$, $h(S) - h(S \oplus O) \leq h(S) =$
782 $\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2$. (An earlier version of this step used $\|x\|^2 - \|Gx\|^2 \leq \|x - Gx\|^2$,
783 which is false, e.g. for $G = \frac{1}{2}I$; the direct bound is sharper by a factor 4.)

784 To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$\begin{aligned}
\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 - \left\| (I - C_{O_i} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 \\
&\geq \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 \\
&= \left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right)
\end{aligned}$$

785 If $h(S) = 0$ then, by prefix monotonicity, every term above vanishes and the weak-supermodularity
786 inequality holds for any $\alpha \geq 1$; assume $h(S) > 0$. The lower bound just obtained is positive because
787 $T - \sum_i \rho_{O_i}^2 > 0$ by hypothesis (this is the only place where a strict contraction is needed; $\rho_j \leq 1$
788 suffices everywhere else). Finally,

$$\begin{aligned}
\frac{h(S) - h(S \oplus O)}{\sum_{i=1}^T (h(S) - h(S \oplus O_i))} &\leq \max \left\{ \frac{\left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2}{\left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(0)}) \right\|_2^2 (T - \sum_{i=1}^T \rho_{O_i}^2)}, 1 \right\} \\
&= \max \left\{ \frac{1}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}
\end{aligned}$$

789 **Postfix monotonicity.** Let $S' = S \oplus N$ and suppose that $I - C_j \mathcal{L}_h^a = P \Lambda_j P^*$ for all j with a
790 common unitary P and $|\lambda_{ji}| \leq 1$ (the products below are of $|\lambda_{ji}|^2$ and $|z_i|^2$; we write them without
791 moduli for the real case). By Theorem A.2,

$$h(S') = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \leq \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} = h(N),$$

792 since every factor $\lambda_{ji}^{2m_j(S)}$ lies in $[0, 1]$. The shared eigenbasis cannot be replaced by invertibility of
793 the individual maps: for

$$A = \begin{bmatrix} 0.0674240 & -0.1864331 \\ -0.1473150 & -0.8707252 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7575738 & 0.4816284 \\ -0.1369844 & 0.3257283 \end{bmatrix}, \quad x = \begin{bmatrix} 0.2812107 \\ -0.5538228 \end{bmatrix},$$

794 both matrices are invertible with spectral norm 0.9, yet $\|BAx\|_2 = 0.330 > 0.225 = \|Bx\|_2$, i.e.,
795 $h((A) \oplus (B)) > h((B))$. (An earlier version of this proof bounded $\|G^{-1}\|$ by the reciprocal of the
796 Lipschitz constant of G , which is not valid.)

797

□

798 A.4 Proof of Theorem A.2

799 **Lemma A.2.** Let $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ where P is a unitary matrix and $\Lambda_j =$
800 $\text{diag}(\lambda_{j1}, \dots, \lambda_{jN})$. If $P^{-1} e_h^{(0)} = z$, then the following equality holds (written for a real or-
801 thogonal P and real eigenvalues; in general z_i^2 and $\lambda_{ji}^{2m_j}$ are to be read as $|z_i|^2$ and $|\lambda_{ji}|^{2m_j}$ here and
802 in the proofs of Propositions 4.2 and 4.3, which use only $\|Px\| = \|x\|$ and $|\lambda_{ji}| \leq 1$):

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \tag{11}$$

803 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned}
h(S) &= \left\| \prod_{t=|S|}^1 (I_N - C_{S_t} \mathcal{L}_h^a) e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^1 (P \Lambda_{S_t} P^{-1}) e_h^{(0)} \right\|^2 \\
&= \left\| P \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\
&= \sum_{i=1}^N z_i^2 \prod_{t=|S|}^1 \lambda_{S_t i}^2 \\
&= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

804

□

805 A.5 Proof of Theorem 4.3

806 **Proposition 4.3.** *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common*
807 *unitary matrix P . Then, h is supermodular.*

808 *Proof.* Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}$$

809 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j . Recall that h is
810 considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

811

$$\begin{aligned}
h(S) - h(S \oplus \omega) &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^N z_i^2 \lambda_{\omega i}^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\
&= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

812 Similarly,

$$\begin{aligned}
h(S') - h(S' \oplus \omega) &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S')} \\
&\stackrel{(a)}{=} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2(m_j(S) + m_j(B))} \\
&\stackrel{(b)}{\leq} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\
&= h(S) - h(S \oplus \omega)
\end{aligned}$$

813 (a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|+1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j} +$
814 $\sum_{t=1}^{|B|} \mathbf{1}_{B_t=j} = m_j(S) + m_j(B)$, (b) since $|\lambda_{ji}| \leq 1$ (as $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ and P is unitary)
815 and $m_j(B) \geq 0$ for all $j \in [K]$
816 □

817 B Proofs for Section 5

818 B.1 Proof of Theorem B.1

819 **Lemma B.1.** *For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r , any*
820 *$a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, the following equality holds true:*

$$\begin{aligned} l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \end{aligned}$$

821 *Proof.* Note that $\sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K-1$

$$\begin{aligned} l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad + (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\ &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad + \sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\ &\quad - (K-1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \end{aligned}$$

$$\begin{aligned}
&= \sum_{j=1}^K \left(\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K-2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \\
&= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K-2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2
\end{aligned}$$

□

824 B.2 Proof of Theorem B.2

825 **Proposition B.2.** *Let $K \geq 2$. For any router r defined by $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$,
826 any $a_h \in \mathcal{A}$, $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:*

$$\log(2) l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)$$

827 *Proof.* By Theorem B.1, we know that

$$\begin{aligned}
\log(2) l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) &= \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - \log(2) (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \\
&\leq \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\stackrel{(a)}{\leq} - \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \\
&= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
\end{aligned}$$

828 (a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \leq 0.5$ (with equality only at a tie for the largest
829 score), which implies that $-\log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$

□

831 B.3 Proof of Theorem 5.1

832 **Theorem 5.1.** *Let $K \geq 2$ and let the per-step costs be normalised state by state, $\hat{c}_j = \tilde{c}_j / \sum_{k=1}^K \tilde{c}_k$,
833 at every state at which $\sum_k \tilde{c}_k > 0$, i.e., at which at least one solver does not annihilate the error
834 (states at which every solver annihilates it carry no routing loss and are discarded), and let $\hat{l}_{\text{route}}$
835 and $\hat{\Psi}$ denote the routing loss and the surrogate with $\tilde{c}_j$ replaced by $\hat{c}_j$; the normalisation does
836 not change the Bayes-optimal decision equation 9 at any state. Then, for any collection of solvers
837 $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, $\hat{\Psi}$ is a Bayes consistent surrogate for $\hat{l}_{\text{route}}$.*

838 *Proof.* Fix a state $x = (a_h, f_h, u_h^{(t)})$ at which $\sum_k \tilde{c}_k > 0$ and write $\hat{c}_j = \tilde{c}_j / \sum_k \tilde{c}_k$, so that
839 $\hat{c}_j \in [0, 1]$ and $\sum_j \hat{c}_j = 1$; let $K \geq 2$. Set $\hat{B}_j = \sum_{k \neq j} \hat{c}_k = 1 - \hat{c}_j$ and $q_j = \hat{B}_j / (K - 1)$; then q is
840 a probability vector on $[K]$, and $\arg \max_j q_j = \arg \min_j \hat{c}_j$ is the Bayes decision equation 9. Let
841 $p = \text{softmax}(\mathbf{g}(x))$.

842 *Surrogate excess.* By the definition equation 10 of the surrogate with the weights $\hat{c}$, $\hat{\Psi}(\mathbf{g}, x) =$
843 $-\sum_j \hat{B}_j \log p_j = (K - 1) \text{CE}(q, p) = (K - 1) [H(q) + \text{KL}(q \| p)]$, with H the entropy and KL
844 the Kullback–Leibler divergence. Since the scores are unrestricted measurable functions, $\hat{\Psi}^*(x) =$
845 $\inf_{\mathbf{g}} \hat{\Psi}(\mathbf{g}, x) = (K - 1) H(q)$ (attained at $p = q$ when $q > 0$, approached in the limit otherwise), so
846 the conditional surrogate excess is $\hat{\Psi}(\mathbf{g}, x) - \hat{\Psi}^*(x) = (K - 1) \text{KL}(q \| p)$.

847 *Routing excess.* With $r(x) = \arg \max_j g_j(x) = \arg \max_j p_j$ and $j^* \in \arg \max_j q_j$, $\hat{l}_{\text{route}}(r, x) =$
848 $\hat{c}_{r(x)} = 1 - (K - 1) q_{r(x)}$ and $\hat{l}_{\text{route}}^*(x) = 1 - (K - 1) q_{j^*}$, so the conditional routing excess is
849 $(K - 1) (q_{j^*} - q_{r(x)})$. Since $p_{r(x)} \geq p_{j^*}$,

$$q_{j^*} - q_{r(x)} \leq (q_{j^*} - p_{j^*}) + (p_{r(x)} - q_{r(x)}) \leq \|q - p\|_1 \leq \sqrt{2 \text{KL}(q \| p)}$$

850 by Pinsker’s inequality, hence

$$\hat{l}_{\text{route}}(r, x) - \hat{l}_{\text{route}}^*(x) \leq (K - 1) \sqrt{2 \text{KL}(q \| p)} = \sqrt{2(K - 1) [\hat{\Psi}(\mathbf{g}, x) - \hat{\Psi}^*(x)]}.$$

851 *Risks.* Taking expectations over the state distribution and using Jensen’s inequality for the concave
852 square root,

$$\mathcal{R}_{\hat{l}_{\text{route}}}(r_n) - \mathcal{R}_{\hat{l}_{\text{route}}}^* \leq \sqrt{2(K - 1) (\mathcal{R}_{\hat{\Psi}}(\mathbf{g}_n) - \mathcal{R}_{\hat{\Psi}}^*)},$$

853 where $\mathcal{R}_{\hat{l}_{\text{route}}}^* = \mathbb{E}[\hat{l}_{\text{route}}^*]$ and $\mathcal{R}_{\hat{\Psi}}^* = \mathbb{E}[\hat{\Psi}^*]$ because the infima are over all measurable functions.
854 Hence $\mathcal{R}_{\hat{\Psi}}(\mathbf{g}_n) - \mathcal{R}_{\hat{\Psi}}^* \rightarrow 0$ implies $\mathcal{R}_{\hat{l}_{\text{route}}}(r_n) - \mathcal{R}_{\hat{l}_{\text{route}}}^* \rightarrow 0$, which is Bayes consistency. (An earlier
855 version of this proof bounded the excess through the calibration function Γ of Mao et al. [2023],
856 whose inverse is defined only on $[0, \log 2]$ while the surrogate excess is unbounded, and assumed
857 uniform absolute bounds on the unnormalised costs, which cannot hold along a convergent solve; the
858 argument above needs neither.) $\square$

859 C Training Details

860 The periodic experiments of Section 7 use the corrector and the lightweight router described in
861 Section E.1; the data distribution below is shared by all experiments, and the DeepONet and LSTM
862 router described afterwards are those of the original submission’s pipeline, whose Dirichlet experiment
863 has been withdrawn (Section D); they are kept for the record.

864 Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random
865 Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ are sampled
866 as:

$$\begin{aligned} \alpha &\sim \text{Log-Uniform}(0.01, 100) \\ \beta &\sim \text{Log-Uniform}(0.1, 1000) \\ \gamma &\sim \text{Uniform}(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\}) \end{aligned}$$

867 Given (α, β, γ) , samples are generated in the Fourier space. For each non-zero frequency mode
868 k , we draw an independent complex coefficient from a Gaussian distribution with mean 0 and
869 variance $\alpha(4\pi^2 \|k\|_2^2 + \beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the
870 zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier
871 Transform to obtain the field in physical space.

872 For each sample, we compute reference solutions with a least squares solver and treat them as ground
873 truth.

874 This data is used to trained our DeepONet models and LSTM routers. All the models were imple-
875 mented using PyTorch and all the models were trained on one Nvidia A40 GPU.

876 Table 2 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train.
877 We then use the model with the best validation loss.

Table 2: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

878 The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$,
879 we also supply the routers with the iteration index t , current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing
880 decisions from the previous iteration. While the iteration index and residual are deterministic
881 functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous
882 routing decisions is a mild deviation from the idealized setting. Nevertheless, this modification is
883 motivated by the same considerations that justify the use of scheduled sampling: under deployment,
884 routing decisions are predicted autoregressively and influence future inputs, whereas training under
885 teacher forcing assumes independence from past predictions. Including previous routing decisions
886 bridges the gap induced by exposure bias by providing the model with trajectory-level information.

887 All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-
888 forcing probability $p_{tf}(e) = ss_{start}$. After the warm-up, the p_{tf} decays geometrically by a factor of
889 $\gamma_{tf} < 1$ per epoch and is floored by ss_{end} :

$$p_{tf}(e) = \begin{cases} ss_{start} & e \leq e_w \\ \max(ss_{start}\gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

890 At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we
891 feed the router’s own predicted iterate.

892 Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated
893 backpropagation through time (TBPTT) [Mozier, 2013, Robinson and Fallside, 1987, Werbos, 1988]:
894 the forward pass unrolls the entire trajectory, but gradients are propagated only through the most
895 recent $w_{bptt}(e)$ steps at epoch e . Hidden states are passed forward between segments, while earlier
896 segments are treated as stop-gradient.

897 We employ a curriculum learning approach analogous to scheduled sampling. Let T_{max} be the
898 horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{bptt}(e) = \begin{cases} w_{start} & e \leq e_w \\ \min\left(T_{max}, w_{start}\gamma_{bptt}^{\lfloor \frac{e-e_w}{f_{bptt}} \rfloor}\right) & e > e_w \end{cases} \quad (12)$$

899 so the window grows geometrically by a factor of $\gamma_{bptt} > 1$ every f_{bptt} epochs and is capped at the
900 full trajectory length.

901 Table 3 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver
902 experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver
903 Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best
904 validation loss for testing. The architecture and dataset size is larger for the Solver ensemble
905 experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 3: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95	0.95
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

The original submission’s Dirichlet experiment (31×31 grid, LSTM router, MLP-branch DeepONet), which this appendix previously kept, has been withdrawn. A review of its evaluation code found that HINTS’ predictions had been mean-centred before comparison with the (uncentred) Dirichlet reference solution while the other methods’ predictions had not, which penalises HINTS by an amount we cannot reconstruct without the original per-instance predictions; in addition, its symmetric-GS rows used the mis-scaled SSOR reference noted in Section E.1. The evaluation code has been corrected (`results.py`), but the experiment has not been rerun, so the wall-clock study of Section E, which is restricted to periodic boundary conditions, is the paper’s only experimental evidence. Extending the corrector to Dirichlet boundaries requires a sine-transform trunk in place of the Fourier trunk and is left to future work.

E Cost-Aware Routing and Wall-Clock Evaluation

This appendix collects the complete wall-clock study that Section 7 summarises: the *wall-clock time to a target accuracy* (charged replay time, Section E.1) of the learned router against HINTS, the classical solver alone and the classical methods a practitioner would reach for on these problems (geometric multigrid and preconditioned Krylov methods), for all five classical solvers and three PDEs, on 128^2 , 256^2 and 512^2 grids, using the cost-aware instantiation of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion equations of Section 7 we add *variable-coefficient diffusion* (Section E.5), on which no Fourier diagonalisation exists and the corrector must be learned from data, and *anisotropic diffusion*, $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ (diffusion tensor $\text{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard model problem on which point-relaxation methods fail to smooth: an error component oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep, and geometric multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-coarsening would be needed). Section E.1 describes the protocol; Sections E.2 to E.5 collect all results per equation (every grid, every tolerance, every baseline, with significance tests and seed trials); Section E.7 studies ensembles of one, two and three classical solvers; Section E.8 accounts for overheads and training cost; and Section E.10 verifies numerically that the assumptions of Theorem 4.1, Propositions 4.2–4.3 and Theorem 5.1 hold in these settings.

E.1 Protocol

Fast, matched implementations. A wall-clock comparison is meaningful only if the classical baseline is not handicapped by implementation. All classical operations are therefore implemented as compiled, single-threaded C kernels on the periodic 5-point stencil (`stencil.c`, built with `cc -O3`):

the residual together with its norm (the stopping test, in one pass), fused (damped) Jacobi sweeps, in-place lexicographic Gauss–Seidel, SOR and symmetric SOR sweeps (forward and backward), a lexicographic line Gauss–Seidel sweep in which every grid line along the strongly coupled axis is solved exactly by a cyclic Thomas algorithm (the line relaxation that anisotropic problems call for), and the full-weighting and bilinear transfers of geometric multigrid, whose V(2,2)-cycles use Gauss–Seidel smoothing—or line Gauss–Seidel smoothing for the anisotropic equation—with rediscrretised coarse operators and a direct solve on the 32×32 coarsest grid (measured on one 128^2 instance, first cycle and asymptotic rate: 0.03 and 0.08 on Poisson with point smoothing, 0.005 and 0.04 on the anisotropic equation with line smoothing, against 0.25 and 0.86 with point smoothing there). At 128^2 a residual evaluation with its norm costs $14 \mu\text{s}$, a Gauss–Seidel sweep $61 \mu\text{s}$, a V-cycle $326 \mu\text{s}$ and one corrector call $672 \mu\text{s}$, all measured in the exact per-iteration form in which the benchmark charges them (Table 7); an earlier version of this study used numpy/scipy sweeps that were $4\text{--}5\times$ slower and correspondingly overstated the router’s margins (Section E.9). The compiled kernels agree with the numpy/scipy implementations they replace to 2×10^{-14} (relative), the line sweep agrees with a sparse block-triangular solve to 10^{-13} , and their operators and one-step iterates match the dense reference implementations of the original submission to the reference’s own precision (relative error below 10^{-7} , the reference assembling its matrices in single precision; 10^{-12} where the reference is exact); the comparison is an assertion in `validate_fast_pde.py`. In the course of this check we found that the original dense symmetric-GS reference mis-implemented the SSOR scaling (the factor $\omega/(2 - \omega)$ was added to D^{-1} instead of multiplying it); it has been corrected, and the wall-clock study uses the standard SSOR throughout. Reference solutions are computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of the discrete system, so true errors are available for evaluation at no charged cost; the FFT solve is also reported as a baseline in every table (it is a direct method that exists because these test problems have constant coefficients and periodic boundaries; at 128^2 the router takes $2.98\times\text{--}3.34\times$ times as long as it, and the variable-coefficient problem of Section E.5 has no such solve). All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch, one thread). The benchmark guards against machine drift, which on this machine is caused by macOS background services (media analysis and Spotlight indexing, which run when the machine is otherwise idle and slow the memory-bound corrector call by up to $3\times$ for minutes at a time): before the timed replays of every test instance it re-times a reference operation (one corrector call) and waits, in 5 s steps and for up to 5 min, while that time is more than 10% slower than the fastest state observed so far in the session; an instance whose reference is still too slow after the wait is timed anyway and re-timed at the end of the cell, after waiting up to 15 min more (the Poisson and convection–diffusion 128^2 cells were produced before this in-cell pass existed, and every one of their instances is re-timed in a separate pass at the end of the study). The guard is a per-instance diagnostic; it does not certify that no drift occurred within a replay. Over the 768 instance timings of the study the guard waited 2211 times, **[pending]** instances were re-timed, **[pending]** remained above the tolerance (largest accepted deviation 222.6%), and the per-instance ratios and load averages are stored with the results (an earlier confirmatory run had been contaminated by a 3.5 h stall of the machine inside one cell, which the harness could not detect). Two accounting checks are reported in Section E.8: the charged time of a router solve is 99–99% of the outer elapsed time of its replay, and its live time to h^2 is 0.99–1.22 times its work-unit time.

Corrector. The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}_h^a u_h^{(t)}$ and evaluated on a 64×64 sensor grid, i.e., on the coarser grid obtained by band-limited (FFT) restriction to the modes $|k_1|, |k_2| \leq 31$ at every grid size (for the anisotropic equation the sensor grid is $N \times N/4$, i.e., full resolution along x_1 , so that the band contains every mode the point smoothers cannot damp; the sensor grid is the only PDE-specific hyperparameter of the pipeline; the variable-coefficient equation uses the isotropic 64×64 grid); its output is prolonged back to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction and never injects error into the high-frequency band that the classical smoother owns (Figure 2). The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it is fit by least squares on 32,000–64,000 exact (residual, error) pairs of the discrete operator drawn from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and reaches a validation relative error of $\approx 2 \times 10^{-6}$ on every mode of the band. The corrector is applied scale-equivariantly, $C_{\text{NO}}(r) = \|Rr\| \mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\text{NO}}(0) = 0$ (Proposition 4.2). Its deployed form is therefore a linear, Fourier-structured map—a least-squares approximation of the exact band-limited

inverse $P(\mathcal{L}_h)_{\text{band}}^{-1}R$ of the discrete operator. Replacing it by that exact inverse changes nothing (development run): on the GS pairing at 128^2 the cost-aware oracle’s median work-unit time to 10^{-8} was 4.472 ms with either corrector and its decision sequence identical on 13 of 16 instances. The gains reported below are thus those of an accurate band-limited spectral correction applied at the right moments; the learned corrector stands in for a spectral inverse that, on these constant-coefficient periodic problems, could also be computed directly. Its cost is dominated by a 4096×4096 matrix–vector product and is therefore almost independent of N ($672 \mu\text{s}$ at 128^2 , [pending] at 512^2 , Table 31), i.e., about 32 Jacobi or 11 GS sweeps at 128^2 and [pending] Jacobi sweeps at 512^2 (Tables 7, 13 and 19). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j are measured once per equation and grid on the benchmark machine, for all operations of the study in one interleaved session, in the exact form in which the timed replay charges them—the update given the current residual, then the residual of the new iterate and its norm, i.e. the stopping test—as the median over eleven interleaved blocks of the block median of 100 repetitions (the minimum and the spread across blocks are stored with the costs); every pairing and ensemble of that grid therefore shares one corrector cost and one calibration. The unit of cost u is one corrector call; every operation is applied $m_j = \max\{1, \text{round}(u/c_j)\}$ times per decision, and these macro-actions are compared by their plain error, i.e. the deployed rule is Algorithm 1 on macro-actions; only an operation dearer than the unit ($c_j > u$, hence $m_j = 1$) is compared per unit of cost, $(\|e_j\|/\|e^{(t)}\|)^{u/c_j}$, and the surrogate weights $\tilde{c}_j$ use the same exponent. Which actions are dearer than the unit is read off the cost tables: at 128^2 none is, so every 128^2 cell runs exactly Algorithm 1 on macro-actions (the rule the exhaustive check of Section E.10 verifies); on the larger grids only a multigrid cycle can be dearer than the unit ([pending]), and in the granularity ablation of Section E.6 the corrector itself is. Tables 7, 13 and 19 list c_j , u and m_j for every cell; the equalisation is exact up to rounding ($m_j c_j / u$ lies within $1/(2m_j - 1)$ of 1). Ensembles reuse the cached pairwise costs of their members, so every member has the same macro-action size in the ensemble as in its pairwise run. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* rows run the cost-agnostic Algorithm 1 on single operations, as in the main text. (An earlier version of this study applied the exponent $u/(m_j c_j)$ to every action, including the rounding error of cheap ones, which is not Algorithm 1; the deployed rule was changed so that the theory covers it.)

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing $\approx 5 \mu\text{s}$ per decision in a warm loop and $16\text{--}20 \mu\text{s}$ in situ, where every decision after a macro-action finds cold caches (measured inside the timed replays, Section E.8), independently of N . It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε -exploration (probability 0.1), followed by two to three DAgger rounds [Ross et al., 2011] in which the current router’s own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 32–128 instances per round (fewer on the larger grids) drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing is trained in seconds to a few minutes. Ensemble routers, which must additionally learn to choose among classical members, use a larger imitation budget (256 rollouts at 128^2 and 128 at 256^2 , six DAgger rounds, hidden width 128, 300 epochs): with the pairwise recipe the router over $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ imitated its oracle imperfectly at tight tolerances (convection–diffusion, 128^2 , $\varepsilon = 10^{-8}$: 3.34 against the oracle’s 2.67 work-unit ms), which the larger budget removes (2.70). At 512^2 the routers are trained from 32 oracle rollouts of at most 400 iterations (stopping error 10^{-8}); for the convection–diffusion Jacobi pairing, where the first such router missed h^2 within the cap on 4 of 16 instances, the reported router uses 64 rollouts of at most 800 iterations—the only per-cell change to the recipe.

Baselines and metric. HINTS applies the corrector every τ -th iteration, first at iteration τ (Section 3). Zhang et al. [2024] use the proportion $1/15$ (one DeepONet call per 14 relaxation sweeps) in all of their two- and three-dimensional experiments and $1/25$ only in the one-dimensional Poisson experiment; since all our problems are two-dimensional, the reference schedule is $\tau = 15$, and $\tau = 25$ is reported as well (an earlier version of this study used it as the reference; the corresponding

margins are $0.96 \times - 4.73 \times$ at h^2 and $1.10 \times - 2.72 \times$ at 10^{-8} on the 128^2 pairwise cells). We further report the *best fixed schedule*, selected *per cell and per tolerance* on the test instances themselves (an optimistic baseline), from the family of HINTS with $\tau \in \{2, 5, 10, 15, 25, 50\}$, phase-shifted HINTS with the first call at iteration 0 and $\tau \in \{5, 10, 15, 25, 50\}$, and the *one-shot schedule*—the corrector once at iteration 0, then the classical solver only—which is the static rule closest to what the router learns at truncation level and therefore the natural ablation for state-dependent routing ($\tau = 1$, the corrector at every iteration, never reduces the error outside the corrector’s band and is omitted); and the best of three *residual-decay rules*, deterministic controls that decide at the router’s granularity from the same residual norm that the stopping test computes: the corrector at iteration 0, then the solver’s macro-action, and the corrector again whenever the per-iteration contraction of the residual over the last macro-action was worse than $\theta \in \{0.9, 0.95, 0.98\}$ (the threshold is selected per cell and tolerance on the test set, like the best fixed schedule); like the router, the rule is executed live in the timed replay, its decision statistic and threshold test are charged, and its decisions are verified against the untimed trace. The classical methods without corrector run on the same stencil, with the same accounting, *inside the same replay loop as the policies*—same instance, same warm-up, same random order, same three replays: multigrid V(2,2) alone; conjugate gradients, CG preconditioned by symmetric GS and by a symmetric V-cycle (forward and backward GS, both compiled) for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles and is not run at 512^2); point-smoothed and line-smoothed multigrid, CG and its two preconditioned variants for anisotropic diffusion; and the sparse direct solve with a cached factorisation together with the multigrid and CG variants for variable-coefficient diffusion. The Krylov methods are SciPy implementations driven by the compiled matrix–vector product and preconditioners: their per-iteration true errors come from an untimed pass with a callback, and the timed pass runs exactly the same number of iterations with a timestamp-only callback (a few hundred nanoseconds per iteration) and verifies that it made exactly that many iterations and that its final iterate reaches the error the untimed pass recorded to within 10% (largest ratio of the two final errors over the study: 1.00; runs failing either check are excluded from the comparisons and counted: **[pending]**). An earlier version of this study timed the classical baselines in separate sessions; the same computation differed by 4–9% between sessions on identical instances (paired, $p < 10^{-4}$), so every cross-session p -value was uninformative, which is why all baselines are now timed in-session. All configuration choices of the study were made on a development test set (GRF seed 72, with the earlier numpy kernels, the earlier forcing sampler and $\tau = 25$ as reference; archived as `results_dev/`); the results reported here are a single confirmatory evaluation of the frozen configurations on a fresh test set (seed 73) with three timed replays per instance, and Section E.9 places the two side by side. The test forcings are drawn from the hierarchical GRF of Section 7 as real white noise filtered in Fourier space, which realises exactly the stated covariance with the Hermitian symmetry of a real field (the development runs used a sampler that drew independent complex coefficients on the half-spectrum, which halved the variance of the modes on the $k_2 = 0$ line); the Nyquist modes are excluded, so every forcing is a trigonometric polynomial with $|k| < N/2$ in both directions, and the instances are drawn sequentially so that the first n instances of any script are the benchmark’s. For each test instance (64 at 128^2 ; 32 at 256^2 , except 16 for anisotropic diffusion; 16 at 512^2) we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual and its norm, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). The reported time to a tolerance is the charged time at the first iteration whose true error is below it, identically for every method: it is a benchmark of the work needed to reach an error level, not of a deployed stopping rule (which would stop on the residual norm that every method computes anyway). We report the median first time at which the error drops below a tolerance ε ; paired speedups—the median over instances of the per-instance time ratio, which is the estimand behind every speedup and test in the speed tables and in Table 1 (the time tables show medians of raw times for scale, which can order differently from the paired median on small samples, and their bold marks compare those raw medians within a pairing)—with bootstrap 95% intervals (2000 resamples over instances) and two-sided Wilcoxon signed-rank tests on the paired log time ratios (the direction is read from the median ratio; “slower” marks a median below one); the study performs several hundred such tests, so the speedup tables mark as significant only comparisons that survive a Holm correction at level 0.05 over all tests of the table. A run that does not reach ε within the iteration cap ($60,000$ at 128^2 , $40,000$ at 256^2 , $8,000$ at 512^2 ; $5,000$ units for the classical baselines, $3,000$ at 512^2) is marked $\dagger$ with the number of censored instances; a median time reported over a majority of censored runs is the median of the mixed vector

1112 of reached times and times-to-cap, a lower bound, and is marked $\geq$. In the paired comparisons a
 1113 censored baseline run enters at its time-to-cap, so that the ratio is a lower bound on the true ratio
 1114 (marked $\geq$ when most baseline runs are censored and the router has no failure); a censored router
 1115 run enters as a ratio of 0 and as the most extreme negative rank in the test, i.e. it counts against
 1116 the router, and is marked with a dagger; a pair in which both are censored is a tie (ratio 1, dropped
 1117 from the test) that carries no information about the true ratio and is marked with a double dagger;
 1118 the classes are decided from the censoring indicators before any substitution. This is a labelled
 1119 capped-time analysis rather than a survival analysis, and the censoring counts are reported with every
 1120 affected cell. Timed replays are repeated three times per instance in a random order over policies
 1121 and baselines, each replay preceded by an untimed warm-up (one corrector call and one sweep, or
 1122 the baseline’s own operation) so that every method starts from the same cache state—the corrector’s
 1123 matrix exceeds the CPU caches, and without the warm-up a policy’s first call was up to 0.1 ms slower
 1124 or faster depending on which policy had run before it; the median over replays is used. The measured
 1125 per-operation costs set the macro-action sizes and define *work units* (measured per-iteration costs
 1126 times executed operations, plus the measured decision cost per decision), a timer-free measure of
 1127 the decision sequence used wherever runs from different sessions must be compared; all reported
 1128 live times are measured directly. We highlight $\varepsilon = h^2$ as a nominal grid-dependent tolerance and
 1129 checked on the confirmatory instances themselves (**[pending]** instances over all equations and grids)
 1130 whether it lies below their discretisation error: comparing the exact discrete solution with the exact
 1131 discrete solution on a four times finer grid for the same forcing (`discretization_error.py`; for
 1132 variable coefficients the coefficient is one grid-independent function), the relative discretisation error,
 1133 estimated as the difference between the two discrete solutions, has median **[pending]**–**[pending]** h^2
 1134 over the equations and grids, and its smallest ratio to h^2 over all instances is **[pending]**; wherever that
 1135 ratio exceeds one, h^2 is a tolerance below the (estimated) discretisation error, and driving the algebraic
 1136 error further is unlikely to improve the computed solution materially (the two error components can
 1137 partly cancel, so “never” would be too strong); we also report 10^{-3} and 10^{-8} , and the iteration-based
 1138 metrics of the main text ($\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\| / \|u_h\|$ over $T = 300$ iterations, compared by paired
 1139 t -tests on $\log \text{AUC}$) on the 128^2 grid. Every router was additionally retrained from scratch with five
 1140 independent seeds (different training instances, exploration and initialisation; the corrector and the
 1141 test set are fixed) and re-run on the test set; because those trials ran in separate sessions, whose live
 1142 timings are not comparable with the main run’s, they are compared in work units.

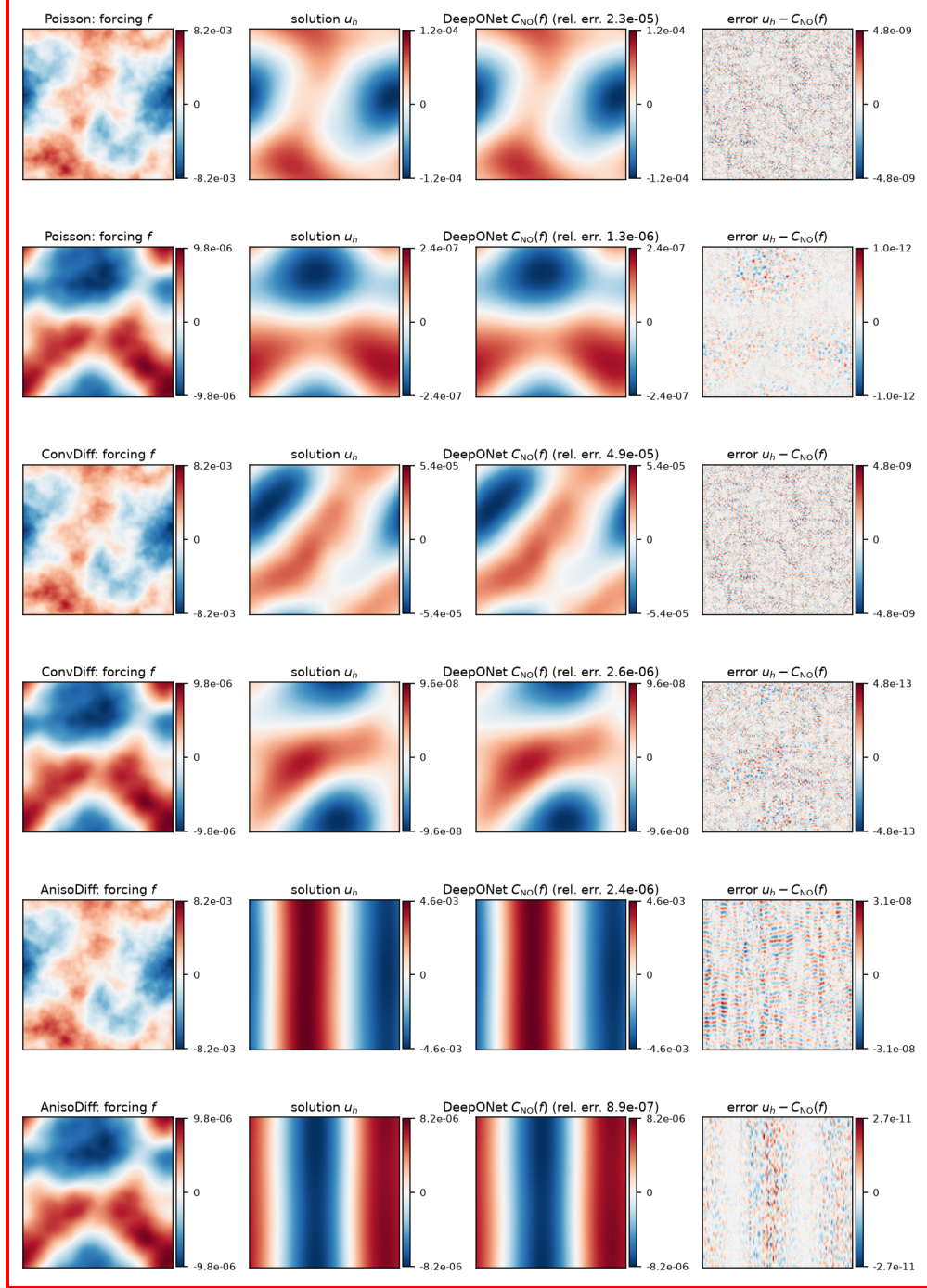


Figure 2: One-shot corrector predictions on the 128×128 grid (two test instances per equation): forcing, exact discrete solution, the DeepONet correction applied to the initial residual ($u_h^{(0)} = 0$), and the remaining error, which is confined to the modes outside the corrector's band (the relative error of the prediction is 10^{-6} – 10^{-5} on the isotropic equations and 10^{-6} on anisotropic diffusion, whose solution is dominated by modes that are smooth in x_2).

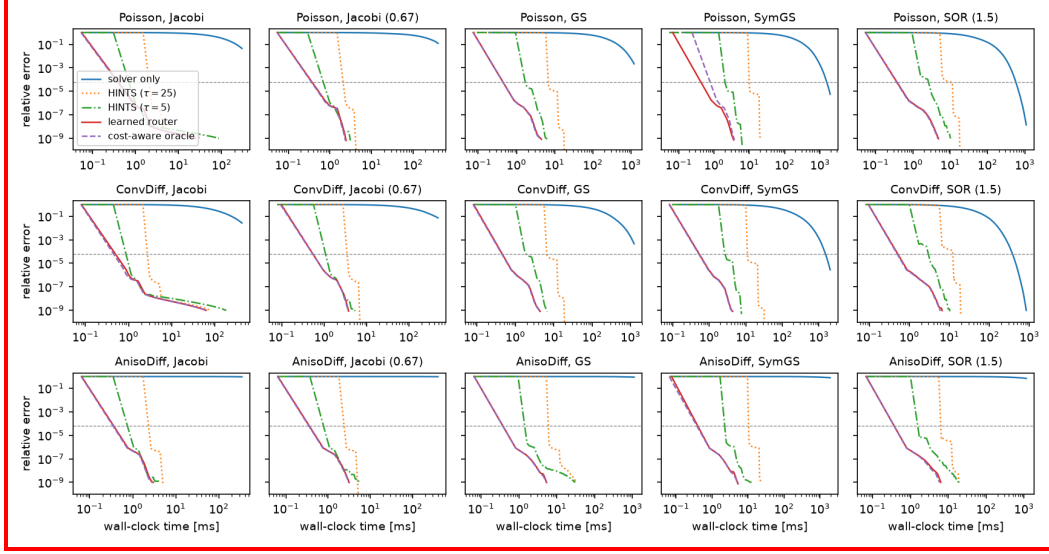


Figure 3: Relative error versus charged wall-clock time on the 128×128 grid for one representative test instance of every pairing (columns) and equation (rows); the dashed line is $\varepsilon = h^2$. The learned router and the cost-aware oracle are indistinguishable.

E.2 Poisson

Table 4 collects the median time to 10^{-3} , h^2 and 10^{-8} for every pairing, every policy and the classical baselines on all three grids; Table 5 the paired speedups of the learned router with their p -values; Figure 4 and table 6 where the iterations go; Table 7 the measured costs and macro-action sizes; Table 8 the iteration-based metrics; and Table 9 the seed trials.

Two regimes. At $\varepsilon = h^2$ the whole solve consists of one corrector call and a few sweeps (Table 6): the router, the cost-aware oracle, phase-shifted HINTS and the one-shot schedule execute the same decision sequence on **[pending]–[pending]**% of the instances (Section 7) and the router’s paired ratio against the best fixed schedule is close to one (Table 5; bold marks the comparisons that remain significant after the Holm correction), while HINTS with its published period, whose first call comes after $\tau - 1$ wasted sweeps, is slower by the factors in the table; the ratio of the solver alone to the router ranges over $0.95 \times$ – $664 \times$ over the 128^2 pairwise cells, the low end being the multigrid pairing, where multigrid alone reaches h^2 in a few cycles ($0.94 \times$ – $0.99 \times$ against the {NO, multigrid} router). Below h^2 the remaining error is the band the corrector cannot see, and the schedules separate: the router calls the corrector again only when the smooth error has re-emerged (two or three calls down to 10^{-8} ; Figure 4), the one-shot schedule never calls it again and pays for hundreds of extra sweeps, and a periodic schedule keeps paying for calls whether or not they help, so at 10^{-8} the paired ratios against HINTS and the one-shot schedule are $0.92 \times$ – $3.20 \times$ and $0.97 \times$ – $52.2 \times$ over the 128^2 pairings, whereas against the best fixed schedule and the best residual-decay rule—both selected on the test set—the outcome depends on the pairing (Table 5; the counts over all pairings are in Section 7, and Section E.6 traces the losses to the macro-action granularity). The cost-agnostic greedy oracle (Algorithm 1 on single operations) makes the same decisions as its cost-aware counterpart on the path to h^2 (they differ later, in when the corrector is called again); the learned router and the cost-aware oracle have the same median iteration and call counts, median times within a few percent, and the same decision sequence to h^2 on **[pending]–[pending]**% of the instances over the 128^2 pairwise cells, and the five retrained routers are compared with the main router in Table 9 **[pending]**.

Classical baselines and larger grids. Against the classical methods without corrector (bottom block of Table 4; timed in the same session as the router of every pairing), the compiled multigrid is the strongest competitor: at 128^2 the paired ratio of multigrid alone to the router (above one favours the router) is $0.95 \times$ – $1.09 \times$ at $\varepsilon = h^2$ and $0.03 \times$ – $0.92 \times$ at 10^{-8} , because one corrector call removes the modes $|k| \leq 31$ at the price of one matrix–vector product whereas a $V(2,2)$ -cycle contracts the

1174 whole spectrum by ≈ 0.03 on the first cycle and ≈ 0.08 asymptotically; routing over $\{\text{NO}, \text{multigrid}\}$
 1175 calls the corrector once and lets the cycles remove the rest ($0.94\times\text{--}0.99\times$ of multigrid alone at h^2 and
 1176 $1.08\times\text{--}1.14\times$ at 10^{-8}), and the ratio of multigrid-preconditioned CG to the router is $0.95\times\text{--}1.64\times$
 1177 at h^2 and $0.03\times\text{--}0.95\times$ at 10^{-8} . The FFT direct solve, exact for this constant-coefficient periodic
 1178 problem, is the reference the router takes $2.98\times\text{--}3.34\times$ times as long as at 128^2 ; the larger grids are
 1179 **[pending]**. At 256^2 and 512^2 **[pending]** the same two regimes are expected to hold with shrinking
 1180 margins for the Jacobi and Gauss–Seidel pairings, because the corrector’s fixed band $|k| \leq 31$ leaves
 1181 the smoother all modes $k \geq 32$, whose slowest member contracts by only 0.98 per Jacobi sweep
 1182 at $N = 512$ (against 0.71 at $N = 128$): every schedule then needs the same ≈ 80 sweeps after
 1183 the first call, the wall-clock time is dominated by sweeps, and the corrector—whose cost is nearly
 1184 grid-independent—is worth fewer and fewer of them, so the best classical method at 512^2 is multigrid,
 1185 alone or with one corrector call. This is the expected limit of a corrector whose band does not grow
 1186 with the grid.

Table 4: Poisson: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids (64, 32 and 16 test instances) for every pairing $\{\text{NO}, \text{solver}\}$ and policy, and for the classical baselines without corrector. “Best fixed schedule” is the best of HINTS ($\tau \in \{2, 5, 10, 15, 25, 50\}$), phase-shifted HINTS ($\tau \in \{5, 10, 15, 25, 50\}$) and the one-shot schedule, and “Best residual-decay rule” the best threshold $\theta \in \{0.9, 0.95, 0.98\}$, both chosen per cell and tolerance on the test set itself (the chosen schedule is the subscript). The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the router where its median is the smallest among the deployable methods of its pairing (solver alone, every fixed schedule, the one-shot and residual-decay rules); the oracles are in italics; $\dagger n$: n instances did not reach the tolerance within the iteration cap ($>$: lower bound).

Pairing	Method	128 × 128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	233.0 ms	329.3 ms	630.1 ms
	HINTS ($\tau=15$)	1.01 ms	1.02 ms	143.9 ms
	Best fixed schedule	703 $\mu\text{s}_{\tau=5, t_0=0}$	718 $\mu\text{s}_{\tau=15, t_0=0}$	66.5 ms _{one-shot}
	Best residual-decay rule	698 $\mu\text{s}_{\theta=0.9}$	715 $\mu\text{s}_{\theta=0.9}$	86.8 ms _{$\theta=0.98$}
	One-shot schedule	708 μs	729 μs	66.5 ms
	Greedy oracle (Alg. 1)	698 μs	717 μs	45.4 ms
	Cost-aware oracle	703 μs	717 μs	46.2 ms
	Learned router (ours)	733 μs	758 μs	46.0 ms
{NO, Jacobi (0.67)}	Solver only	350.5 ms	493.4 ms	938.5 ms
	HINTS ($\tau=15$)	981 μs	981 μs	1.94 ms
	Best fixed schedule	690 $\mu\text{s}_{\tau=10, t_0=0}$	711 $\mu\text{s}_{\tau=5, t_0=0}$	1.74 ms _{$\tau=15, t_0=0$}
	Best residual-decay rule	689 $\mu\text{s}_{\theta=0.95}$	711 $\mu\text{s}_{\theta=0.98}$	2.69 ms _{$\theta=0.9$}
	One-shot schedule	700 μs	717 μs	76.7 ms
	Greedy oracle (Alg. 1)	693 μs	709 μs	1.72 ms
	Cost-aware oracle	686 μs	709 μs	1.99 ms
	Learned router (ours)	725 μs	743 μs	2.09 ms
{NO, GS}	Solver only	308.0 ms	433.3 ms	823.3 ms
	HINTS ($\tau=15$)	1.52 ms	1.58 ms	4.52 ms
	Best fixed schedule	676 $\mu\text{s}_{\tau=10, t_0=0}$	711 $\mu\text{s}_{\tau=15, t_0=0}$	2.02 ms _{$\tau=10, t_0=0$}
	Best residual-decay rule	684 $\mu\text{s}_{\theta=0.98}$	717 $\mu\text{s}_{\theta=0.9}$	2.76 ms _{$\theta=0.9$}
	One-shot schedule	683 μs	718 μs	107.7 ms
	Greedy oracle (Alg. 1)	686 μs	721 μs	2.45 ms
	Cost-aware oracle	687 μs	707 μs	2.06 ms
	Learned router (ours)	721 μs	752 μs	2.14 ms
{NO, SymGS}	Solver only	234.5 ms	329.7 ms	627.3 ms
	HINTS ($\tau=15$)	1.96 ms	1.99 ms	4.01 ms
	Best fixed schedule	676 $\mu\text{s}_{\tau=25, t_0=0}$	726 $\mu\text{s}_{\text{one-shot}}$	1.85 ms _{$\tau=5, t_0=0$}
	Best residual-decay rule	685 $\mu\text{s}_{\theta=0.9}$	720 $\mu\text{s}_{\theta=0.95}$	3.29 ms _{$\theta=0.9$}
	One-shot schedule	685 μs	726 μs	78.1 ms
	Greedy oracle (Alg. 1)	695 μs	737 μs	1.86 ms
	Cost-aware oracle	693 μs	735 μs	2.03 ms
	Learned router (ours)	720 μs	772 μs	2.11 ms
{NO, SOR (1.5)}	Solver only	105.4 ms	148.6 ms	282.7 ms
	HINTS ($\tau=15$)	1.53 ms	3.03 ms	4.56 ms
	Best fixed schedule	682 $\mu\text{s}_{\tau=25, t_0=0}$	711 $\mu\text{s}_{\tau=25, t_0=0}$	2.70 ms _{$\tau=15, t_0=0$}
	Best residual-decay rule	685 $\mu\text{s}_{\theta=0.95}$	721 $\mu\text{s}_{\theta=0.95}$	4.02 ms _{$\theta=0.9$}
	One-shot schedule	688 μs	723 μs	55.0 ms
	Greedy oracle (Alg. 1)	687 μs	732 μs	3.03 ms
	Cost-aware oracle	696 μs	733 μs	2.69 ms
	Learned router (ours)	716 μs	764 μs	2.85 ms
{NO, Multigrid}	Solver only	680 μs	901 μs	1.90 ms
	HINTS ($\tau=15$)	678 μs	933 μs	1.89 ms
	Best fixed schedule	651 $\mu\text{s}_{\tau=15, t_0=0}$	821 $\mu\text{s}_{\tau=25, t_0=0}$	1.65 ms _{$\tau=10, t_0=0$}
	Best residual-decay rule	654 $\mu\text{s}_{\theta=0.98}$	824 $\mu\text{s}_{\theta=0.98}$	1.65 ms _{$\theta=0.95$}
	One-shot schedule	666 μs	822 μs	1.65 ms
	Greedy oracle (Alg. 1)	657 μs	816 μs	1.65 ms
	Cost-aware oracle	672 μs	801 μs	1.67 ms
	Learned router (ours)	684 μs	874 μs	1.70 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	692 μs	1.01 ms	2.01 ms
	CG	4.48 ms	5.41 ms	7.59 ms
	PCG (SymGS)	6.33 ms	9.56 ms	16.5 ms
	PCG (multigrid)	706 μs	978 μs	1.70 ms
	FFT direct solve (exact; constant coefficients only)	246 μs	246 μs	246 μs

Table 5: Poisson: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), over the best fixed period for that cell, over multigrid alone and over multigrid-preconditioned CG, with the two-sided Wilcoxon p -value (“slower” marks a median ratio below one) and the bootstrap 95% interval; bold: paired ratio $\geq 1.1\times$ and significant after a Holm correction over all tests of the table.

Pairing	τ	128 × 128							
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. FFT solve	
{NO, Jacobi}	h^2	1.37 × (1.34, 1.41) ($< 10^{-10}$)	0.96 × [0.95, 0.99] (slower, 0.095)	0.95 × [0.94, 0.97] (slower, 0.020)	0.95 × [0.95, 0.97] (slower, 2×10^{-10})	0.98 × [0.95, 1.05] (slower, 0.952)	0.99 × [0.96, 1.03] (slower, 0.730)	0.32 × [0.31, 0.33] (slower, $< 10^{-10}$)	
	10^{-8}	3.07 × (2.86, 3.34) ($< 10^{-10}$)	1.24 × (1.04, 1.58) (10^{-7})	1.92 × (1.86, 1.96) ($< 10^{-10}$)	1.24 × (1.04, 1.58) (10^{-7})	0.04 × [0.03, 0.11] (slower, $< 10^{-10}$)	0.04 × [0.02, 0.10] (slower, $< 10^{-10}$)	0.01 × [0.00, 0.01] (slower, $< 10^{-10}$)	
{NO, Jacobi (0.67)}	h^2	1.32 × (1.24, 1.37) ($< 10^{-10}$)	0.98 × [0.96, 1.03] (slower, 0.158)	0.95 × [0.94, 0.96] (slower, 9×10^{-10})	0.95 × [0.94, 0.96] (slower, 4×10^{-10})	1.09 × [1.03, 1.15] (3×10^{-10})	1.07 × [1.03, 1.13] (2×10^{-10})	0.34 × [0.31, 0.34] (slower, $< 10^{-10}$)	
	10^{-8}	0.93 × [0.92, 0.94] (slower, $< 10^{-10}$)	0.83 × [0.81, 0.86] (slower, 10^{-10})	1.29 × (1.28, 1.30) ($< 10^{-10}$)	36.9 × (33.1, 41.6) ($< 10^{-10}$)	0.92 × [0.84, 0.96] (slower, 10^{-10})	0.79 × [0.78, 0.81] (slower, $< 10^{-10}$)	0.12 × [0.12, 0.12] (slower, $< 10^{-10}$)	
{NO, GS}	h^2	2.06 × (1.82, 2.14) ($< 10^{-10}$)	0.95 × [0.94, 0.96] (slower, 9×10^{-10})	0.95 × [0.94, 0.96] (slower, 5×10^{-10})	0.94 × [0.93, 0.95] (slower, 10^{-10})	0.98 × [0.96, 1.03] (slower, 0.805)	0.99 × [0.97, 1.03] (slower, 0.820)	0.32 × [0.27, 0.33] (slower, $< 10^{-10}$)	
	10^{-8}	2.06 × (1.84, 2.15) ($< 10^{-10}$)	0.91 × [0.90, 0.92] (slower, 5×10^{-10})	1.26 × (1.26, 1.31) (10^{-10})	40.8 × (34.4, 46.8) ($< 10^{-10}$)	0.79 × [0.71, 0.82] (slower, $< 10^{-10}$)	0.66 × [0.58, 0.74] (slower, $< 10^{-10}$)	0.11 × [0.09, 0.11] (slower, $< 10^{-10}$)	
{NO, SymGS}	h^2	2.53 × (2.38, 2.70) ($< 10^{-10}$)	0.95 × [0.94, 0.96] (slower, $< 10^{-10}$)	0.95 × [0.94, 0.96] (slower, $< 10^{-10}$)	0.95 × [0.94, 0.96] (slower, $< 10^{-10}$)	1.01 × [0.98, 1.06] (0.113)	1.01 × [0.98, 1.04] (0.183)	0.31 × [0.28, 0.33] (slower, $< 10^{-10}$)	
	10^{-8}	1.89 × (1.83, 1.92) ($< 10^{-10}$)	0.86 × [0.84, 0.88] (slower, 2×10^{-10})	1.55 × (1.51, 1.58) ($< 10^{-10}$)	37.8 × (31.2, 54.6) ($< 10^{-10}$)	0.82 × [0.80, 0.84] (slower, $< 10^{-10}$)	0.70 × [0.67, 0.76] (slower, $< 10^{-10}$)	0.11 × [0.11, 0.12] (slower, $< 10^{-10}$)	
{NO, SOR (1.5)}	h^2	3.58 × (2.93, 4.42) ($< 10^{-10}$)	0.96 × [0.95, 0.97] (slower, 10^{-10})	0.96 × [0.95, 0.96] (slower, 3×10^{-10})	0.95 × [0.95, 0.97] (slower, 10^{-10})	0.97 × [0.94, 1.00] (slower, 0.025)	0.95 × [0.91, 1.00] (slower, 0.003)	0.32 × [0.25, 0.34] (slower, $< 10^{-10}$)	
	10^{-8}	1.63 × (1.25, 1.83) ($< 10^{-10}$)	0.98 × [0.97, 0.99] (slower, 0.036)	1.42 × (1.38, 1.50) ($< 10^{-10}$)	18.4 × (14.6, 22.5) ($< 10^{-10}$)	0.66 × [0.57, 0.70] (slower, $< 10^{-10}$)	0.58 × [0.46, 0.64] (slower, $< 10^{-10}$)	0.09 × [0.07, 0.10] (slower, $< 10^{-10}$)	
{NO, Multigrid}	h^2	0.99 × [0.97, 0.99] (slower, 0.003)	0.97 × [0.96, 0.97] (slower, $< 10^{-10}$)	0.96 × [0.96, 0.97] (slower, $< 10^{-10}$)	0.96 × [0.96, 0.97] (slower, 5×10^{-10})	0.99 × [0.98, 1.00] (slower, 0.011)	0.99 × [0.98, 1.00] (slower, 0.022)	0.28 × [0.24, 0.33] (slower, $< 10^{-10}$)	
	10^{-8}	1.14 × (1.10, 1.18) (5×10^{-9})	0.97 × [0.97, 0.98] (slower, 5×10^{-9})	0.97 × [0.97, 0.97] (slower, 3×10^{-10})	0.97 × [0.97, 0.97] (slower, 10^{-10})	1.14 × (1.10, 1.18) (1×10^{-7})	0.97 × [0.90, 0.98] (slower, 10^{-7})	0.14 × [0.13, 0.15] (slower, $< 10^{-10}$)	

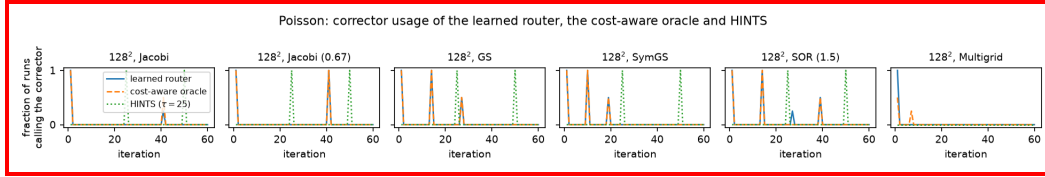


Figure 4: Poisson: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=15$), for every pairing (columns) and grid (rows; all test instances of each grid).

Table 6: Poisson: median iterations (and corrector calls) to $\varepsilon = h^2$ for HINTS ($\tau=15$), the cost-aware oracle and the learned router on every grid.

Pairing	128 × 128		
	HINTS-15	oracle	router
{NO, Jacobi}	15 (1)	2 (1)	2 (1)
{NO, Jacobi (0.67)}	15 (1)	2 (1)	2 (1)
{NO, GS}	16 (1)	2 (1)	2 (1)
{NO, SymGS}	15 (1)	2 (1)	2 (1)
{NO, SOR (1.5)}	30 (2)	2 (1)	2 (1)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)

Table 7: Poisson: measured cost of one classical iteration c_{solver} and of one corrector call c_{NO} (residual evaluation included), and the resulting macro-action size m_{solver} (sweeps per decision; the corrector is always applied once).

Pairing	128×128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	21 μs	672 μs	32
{NO, Jacobi (0.67)}	21 μs	672 μs	32
{NO, GS}	61 μs	672 μs	11
{NO, SymGS}	92 μs	672 μs	7
{NO, SOR (1.5)}	61 μs	672 μs	11
{NO, Multigrid}	326 μs	672 μs	2

Table 8: Poisson, 128×128 : iteration-based metrics of the main text—relative error after $T = 300$ iterations and AUC over the T iterations (mean and standard error over 64 instances; p : one-sided paired t -test that the method’s AUC exceeds the router’s). Bold: router better than both solver-only and HINTS.

Method	AUC	p
{NO, Jacobi}		
Solver only	2.69e+02 (7.2e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (2.7e-03)	$< 10^{-10}$
Learned router (ours)	6.08e-03 (1.5e-03)	-
Cost-aware oracle	6.08e-03 (1.5e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.78e+02 (5.7e-01)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (2.4e-03)	$< 10^{-10}$
Learned router (ours)	2.13e-03 (5.0e-04)	-
Cost-aware oracle	2.13e-03 (5.0e-04)	-
{NO, GS}		
Solver only	2.44e+02 (1.0e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.38e+01 (6.1e-03)	$< 10^{-10}$
Learned router (ours)	1.71e-03 (4.1e-04)	-
Cost-aware oracle	1.71e-03 (4.1e-04)	-
{NO, SymGS}		
Solver only	2.05e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.37e+01 (1.1e-02)	$< 10^{-10}$
Learned router (ours)	1.26e-03 (3.0e-04)	-
Cost-aware oracle	1.26e-03 (3.0e-04)	-
{NO, SOR (1.5)}		
Solver only	1.75e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.35e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	2.79e-03 (6.7e-04)	-
Cost-aware oracle	2.80e-03 (6.7e-04)	-

Table 9: Poisson, 128×128 : five independent router-training seeds per pairing—mean and standard deviation over seeds of the median work-unit time to h^2 , the share of test instances on which the seed router’s decision sequence to h^2 coincides with the main router’s, the range over seeds of the paired speedup over HINTS ($\tau=15$), and the number of seeds significantly faster ($p < 0.01$) than both HINTS ($\tau=15$) and the best fixed period.

[results pending: confirmatory run in progress]

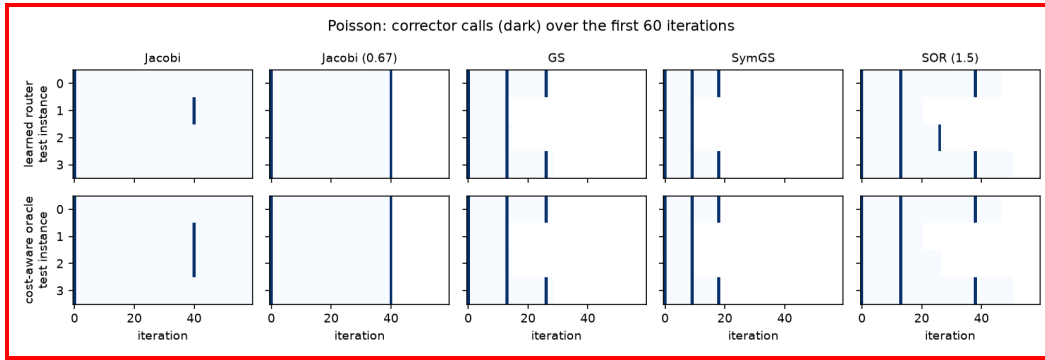


Figure 5: Poisson, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance, for every pairing.

1187 E.3 Convection–diffusion

1188 The convection–diffusion results (Tables 10 to 15 and fig. 6) mirror the Poisson ones: **[pending]**
 1189 under the revised protocol. In the first confirmatory run (revision 1, reference $\tau = 25$) they mirrored
 1190 the Poisson ones, with the nonsymmetric operator changing only the details: the undamped-Jacobi
 1191 pairing stalled at the near-Nyquist band below 10^{-6} for every policy, and GMRES(20) was far slower
 1192 than the router.

Table 10: Convection–diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids for every pairing and policy, and for the classical baselines without corrector (conventions as in Table 4).

Pairing	Method	128×128		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	232.1 ms	328.1 ms	631.4 ms
	HINTS ($\tau=15$)	1.07 ms	1.08 ms	222.9 ms
	Best fixed schedule	$763 \mu s_{\tau=10, t_0=0}$	$781 \mu s_{\text{one-shot}}$	$68.7 \text{ ms}_{\text{one-shot}}$
	Best residual-decay rule	$762 \mu s_{\theta=0.98}$	$798 \mu s_{\theta=0.9}$	$136.6 \text{ ms}_{\theta=0.95}$
	One-shot schedule	$770 \mu s$	$781 \mu s$	68.7 ms
	Greedy oracle (Alg. 1)	$761 \mu s$	$789 \mu s$	67.5 ms
	Cost-aware oracle	$747 \mu s$	$798 \mu s$	65.5 ms
	Learned router (ours)	$806 \mu s$	$833 \mu s$	67.2 ms
{NO, Jacobi (0.67)}	Solver only	349.7 ms	493.7 ms	944.2 ms
	HINTS ($\tau=15$)	1.09 ms	1.09 ms	2.13 ms
	Best fixed schedule	$764 \mu s_{\tau=10, t_0=0}$	$801 \mu s_{\text{one-shot}}$	$1.96 \text{ ms}_{\tau=10}$
	Best residual-decay rule	$765 \mu s_{\theta=0.9}$	$803 \mu s_{\theta=0.9}$	$2.94 \text{ ms}_{\theta=0.95}$
	One-shot schedule	$772 \mu s$	$801 \mu s$	62.6 ms
	Greedy oracle (Alg. 1)	$772 \mu s$	$816 \mu s$	1.93 ms
	Cost-aware oracle	$760 \mu s$	$806 \mu s$	2.23 ms
	Learned router (ours)	$797 \mu s$	$864 \mu s$	2.33 ms
{NO, GS}	Solver only	258.6 ms	366.2 ms	700.5 ms
	HINTS ($\tau=15$)	1.60 ms	1.94 ms	4.78 ms
	Best fixed schedule	$747 \mu s_{\tau=25, t_0=0}$	$799 \mu s_{\tau=50, t_0=0}$	$2.42 \text{ ms}_{\tau=15, t_0=0}$
	Best residual-decay rule	$754 \mu s_{\theta=0.95}$	$812 \mu s_{\theta=0.98}$	$3.00 \text{ ms}_{\theta=0.9}$
	One-shot schedule	$764 \mu s$	$829 \mu s$	140.7 ms
	Greedy oracle (Alg. 1)	$746 \mu s$	$821 \mu s$	2.77 ms
	Cost-aware oracle	$753 \mu s$	$852 \mu s$	2.43 ms
	Learned router (ours)	$784 \mu s$	$885 \mu s$	2.50 ms
{NO, SymGS}	Solver only	227.6 ms	322.4 ms	617.3 ms
	HINTS ($\tau=15$)	2.01 ms	2.11 ms	4.17 ms
	Best fixed schedule	$728 \mu s_{\tau=25, t_0=0}$	$789 \mu s_{\tau=10, t_0=0}$	$2.04 \text{ ms}_{\tau=5, t_0=0}$
	Best residual-decay rule	$720 \mu s_{\theta=0.95}$	$771 \mu s_{\theta=0.9}$	$3.66 \text{ ms}_{\theta=0.9}$
	One-shot schedule	$739 \mu s$	$808 \mu s$	112.3 ms
	Greedy oracle (Alg. 1)	$733 \mu s$	$771 \mu s$	2.54 ms
	Cost-aware oracle	$723 \mu s$	$776 \mu s$	2.16 ms
	Learned router (ours)	$752 \mu s$	$831 \mu s$	2.25 ms
{NO, SOR (1.5)}	Solver only	61.2 ms	86.7 ms	165.0 ms
	HINTS ($\tau=15$)	1.57 ms	3.13 ms	4.70 ms
	Best fixed schedule	$721 \mu s_{\tau=10, t_0=0}$	$815 \mu s_{\tau=15, t_0=0}$	$3.13 \text{ ms}_{\tau=25, t_0=0}$
	Best residual-decay rule	$722 \mu s_{\theta=0.95}$	$834 \mu s_{\theta=0.98}$	$3.66 \text{ ms}_{\theta=0.9}$
	One-shot schedule	$724 \mu s$	$822 \mu s$	46.2 ms
	Greedy oracle (Alg. 1)	$716 \mu s$	$805 \mu s$	3.32 ms
	Cost-aware oracle	$734 \mu s$	$867 \mu s$	3.04 ms
	Learned router (ours)	$750 \mu s$	$888 \mu s$	3.12 ms
{NO, Multigrid}	Solver only	708 μs	1.02 ms	1.97 ms
	HINTS ($\tau=15$)	706 μs	1.02 ms	1.92 ms
	Best fixed schedule	$706 \mu s_{\tau=15}$	$981 \mu s_{\tau=15, t_0=0}$	$1.73 \text{ ms}_{\tau=25, t_0=0}$
	Best residual-decay rule	$725 \mu s_{\theta=0.95}$	$987 \mu s_{\theta=0.9}$	$1.74 \text{ ms}_{\theta=0.98}$
	One-shot schedule	$726 \mu s$	1.00 ms	1.75 ms
	Greedy oracle (Alg. 1)	$730 \mu s$	1000 μs	1.77 ms
	Cost-aware oracle	708 μs	959 μs	1.79 ms
	Learned router (ours)	749 μs	1.04 ms	1.83 ms
<i>Classical baselines without corrector (same stencil, same accounting)</i>				
	Multigrid V(2,2) alone (point GS)	700 μs	1.02 ms	2.02 ms
	BiCGSTAB	18.0 ms	21.7 ms	30.0 ms
	BiCGSTAB (multigrid)	764 μs	1.47 ms	2.18 ms
	GMRES(20)	179.2 ms	255.6 ms	490.3 ms
	FFT direct solve (exact; constant coefficients only)	250 μs	250 μs	250 μs

Table 11: Convection–diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), the best fixed period, multigrid alone and multigrid-preconditioned BiCGSTAB, with two-sided Wilcoxon p -values (conventions as in Table 5).

Pairing	ε	128×128							
		vs. HINTS-15	vs. best schedule	vs. best decay rule	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. FFT solve	
{NO, Jacobi}	h^2	1.37 × [1.29, 1.40] ($< 10^{-10}$)	0.95 × [0.93, 0.97] (slower, 3×10^{-6})	1.00 × [0.98, 1.02] (0.611)	0.95 × [0.93, 0.97] (slower, 3×10^{-6})	0.95 × [0.89, 1.04] (slower, 0.231)	1.57 × [1.18, 1.72] (0.025)	0.29 × [0.25, 0.31] (slower, $< 10^{-10}$)	
	10^{-8}	3.30 × [3.04, 3.56] ($< 10^{-10}$)	1.07 × [1.01, 1.48] (3×10^{-5})	2.62 × [1.96, 2.65] ($< 10^{-10}$)	1.07 × [1.01, 1.48] (3×10^{-5})	0.03 × [0.02, 0.06] (slower, $< 10^{-10}$)	0.03 × [0.02, 0.06] (slower, 3×10^{-10})	0.00 × [0.00, 0.01] (slower, $< 10^{-10}$)	
{NO, Jacobi (0.67)}	h^2	1.26 × [1.18, 1.33] ($< 10^{-10}$)	0.96 × [0.93, 0.98] (slower, 4×10^{-6})	0.95 × [0.93, 0.98] (slower, 4×10^{-6})	0.96 × [0.93, 0.98] (slower, 4×10^{-6})	1.05 × [1.02, 1.13] (0×10^{-10})	1.64 × [1.52, 1.74] ($< 10^{-10}$)	0.31 × [0.26, 0.33] (slower, $< 10^{-10}$)	
	10^{-8}	0.92 × [0.90, 0.94] (slower, $< 10^{-10}$)	0.84 × [0.83, 0.88] (slower, 4×10^{-6})	1.29 × [1.27, 1.31] ($< 10^{-10}$)	27.0 × [25.2, 28.5] ($< 10^{-10}$)	0.85 × [0.79, 0.88] (slower, $< 10^{-10}$)	0.95 × [0.93, 0.98] (slower, $< 10^{-10}$)	0.11 × [0.11, 0.11] (slower, $< 10^{-10}$)	
{NO, GS}	h^2	2.20 × [2.03, 2.36] ($< 10^{-10}$)	0.96 × [0.95, 0.98] (slower, 0.048)	0.97 × [0.95, 1.00] (slower, 0.039)	0.97 × [0.95, 1.00] (slower, 0.145)	0.99 × [0.95, 1.03] (slower, 0.434)	1.51 × [1.41, 1.63] (2×10^{-6})	0.28 × [0.25, 0.31] (slower, $< 10^{-10}$)	
	10^{-8}	1.03 × [1.46, 2.68] ($< 10^{-10}$)	1.03 × [1.01, 1.04] (0.018)	1.31 × [1.28, 1.32] ($< 10^{-10}$)	52.2 × [49.5, 59.6] ($< 10^{-10}$)	0.72 × [0.65, 0.75] (slower, $< 10^{-10}$)	0.87 × [0.65, 0.98] (slower, $< 10^{-10}$)	0.10 × [0.08, 0.11] (slower, $< 10^{-10}$)	
{NO, SymGS}	h^2	2.50 × [2.27, 2.69] ($< 10^{-10}$)	0.96 × [0.95, 0.97] (slower, 10^{-7})	0.95 × [0.94, 0.96] (slower, 10^{-7})	0.96 × [0.95, 0.98] (slower, 3×10^{-6})	1.01 × [0.96, 1.09] (0.857)	1.64 × [1.41, 1.77] (2×10^{-6})	0.30 × [0.27, 0.32] (slower, $< 10^{-10}$)	
	10^{-8}	1.82 × [1.79, 1.86] ($< 10^{-10}$)	0.82 × [0.81, 0.84] (slower, 5×10^{-10})	1.61 × [1.58, 1.63] ($< 10^{-10}$)	40.9 × [39.6, 58.6] ($< 10^{-10}$)	0.74 × [0.70, 0.77] (slower, $< 10^{-10}$)	0.92 × [0.84, 0.94] (slower, 2×10^{-10})	0.11 × [0.10, 0.11] (slower, $< 10^{-10}$)	
{NO, SOR (1.5)}	h^2	3.52 × [2.89, 4.10] ($< 10^{-10}$)	0.96 × [0.95, 0.98] (slower, 10^{-7})	0.96 × [0.96, 0.98] (slower, 0.035)	0.96 × [0.95, 0.98] (slower, 0.020)	0.95 × [0.89, 0.99] (slower, 0.059)	1.45 × [1.11, 1.72] (7×10^{-6})	0.27 × [0.22, 0.32] (slower, $< 10^{-10}$)	
	10^{-8}	1.50 × [1.27, 1.81] ($< 10^{-10}$)	1.24 × [1.22, 1.26] ($< 10^{-10}$)	1.47 × [1.42, 1.51] ($< 10^{-10}$)	13.9 × [12.1, 15.9] ($< 10^{-10}$)	0.60 × [0.55, 0.70] (slower, $< 10^{-10}$)	0.68 × [0.58, 0.81] (slower, $< 10^{-10}$)	0.08 × [0.07, 0.10] (slower, $< 10^{-10}$)	
{NO, Multigrid}	h^2	0.95 × [0.92, 0.97] (slower, 0.023)	0.96 × [0.96, 0.97] (slower, 6×10^{-10})	0.96 × [0.96, 0.97] (slower, 8×10^{-6})	0.96 × [0.95, 0.97] (slower, 4×10^{-6})	0.94 × [0.92, 0.96] (slower, 0.012)	1.31 × [1.26, 1.36] ($< 10^{-10}$)	0.24 × [0.22, 0.31] (slower, $< 10^{-10}$)	
	10^{-8}	1.09 × [0.98, 1.15] (5×10^{-3})	0.96 × [0.95, 0.97] (slower, $< 10^{-10}$)	0.97 × [0.96, 0.97] (slower, 4×10^{-6})	0.97 × [0.96, 0.98] (slower, 10^{-6})	1.08 × [0.98, 1.13] (0.001)	1.16 × [1.07, 1.24] (6×10^{-9})	0.13 × [0.12, 0.14] (slower, $< 10^{-10}$)	

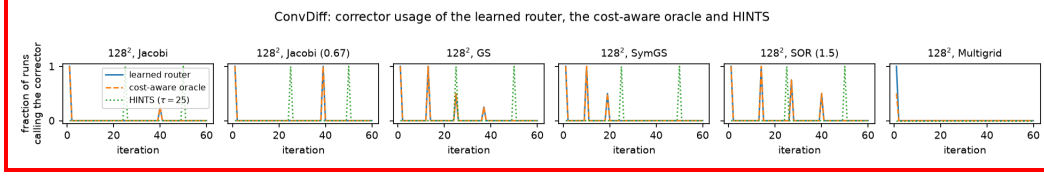


Figure 6: Convection–diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

Table 12: Convection–diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$ on every grid.

Pairing	128 × 128		
	HINTS-15	oracle	router
{NO, Jacobi}	15 (1)	2 (1)	2 (1)
{NO, Jacobi (0.67)}	15 (1)	3 (1)	3 (1)
{NO, GS}	20 (1)	2 (1)	2 (1)
{NO, SymGS}	16 (1)	2 (1)	2 (1)
{NO, SOR (1.5)}	30 (2)	4 (1)	4 (1)
{NO, Multigrid}	3 (0)	2 (0)	2 (1)

Table 13: Convection–diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128		
	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	21 μs	626 μs	30
{NO, Jacobi (0.67)}	19 μs	626 μs	34
{NO, GS}	54 μs	626 μs	12
{NO, SymGS}	81 μs	626 μs	8
{NO, SOR (1.5)}	54 μs	626 μs	12
{NO, Multigrid}	288 μs	626 μs	2

Table 14: Convection–diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

Method	AUC	p
{NO, Jacobi}		
Solver only	2.60e+02 (1.4e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (5.7e-03)	$< 10^{-10}$
Learned router (ours)	1.38e-02 (3.2e-03)	-
Cost-aware oracle	1.38e-02 (3.2e-03)	-
{NO, Jacobi (0.67)}		
Solver only	2.71e+02 (1.1e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.39e+01 (5.1e-03)	$< 10^{-10}$
Learned router (ours)	4.83e-03 (1.0e-03)	-
Cost-aware oracle	4.83e-03 (1.0e-03)	-
{NO, GS}		
Solver only	2.24e+02 (1.8e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.37e+01 (1.4e-02)	$< 10^{-10}$
Learned router (ours)	3.76e-03 (8.1e-04)	-
Cost-aware oracle	3.76e-03 (8.1e-04)	-
{NO, SymGS}		
Solver only	1.89e+02 (2.2e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.35e+01 (2.2e-02)	$< 10^{-10}$
Learned router (ours)	2.89e-03 (6.3e-04)	-
Cost-aware oracle	2.89e-03 (6.3e-04)	-
{NO, SOR (1.5)}		
Solver only	1.21e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=15$)	1.31e+01 (3.6e-02)	$< 10^{-10}$
Learned router (ours)	5.86e-03 (1.3e-03)	-
Cost-aware oracle	5.86e-03 (1.3e-03)	-

Table 15: Convection–diffusion, 128×128 : five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

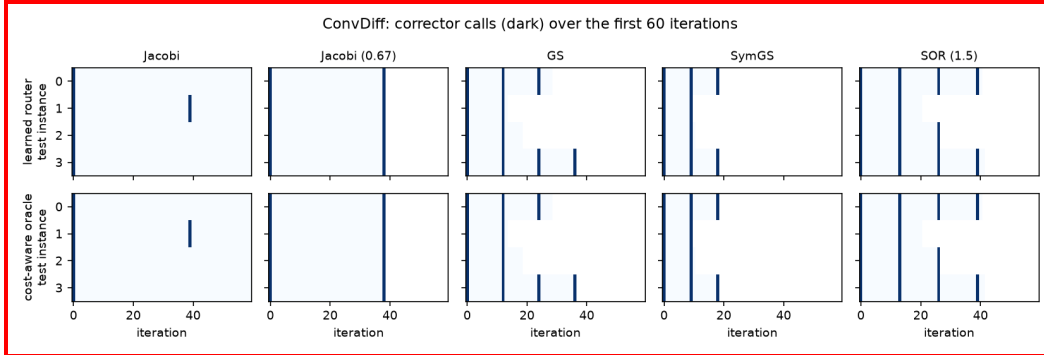


Figure 7: Convection–diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1193 E.4 Anisotropic diffusion

1194 The anisotropic equation is where the two components of the hybrid are most complementary and
 1195 where the point smoothers are weakest: they damp the x_1 -oscillatory modes only by $(1-\epsilon)/(1+\epsilon) \approx$
 1196 0.98 per sweep (in the first confirmatory run none of the five point smoothers alone reached h^2 within
 1197 the iteration cap on either grid; the revised cells are **[pending]**), whereas the corrector, whose
 1198 sensor grid keeps full resolution along x_1 , removes exactly those modes together with the smooth
 1199 ones in a single call and leaves the smoother only the x_2 -oscillatory band, which it damps at the
 1200 isotropic rate. The corrector’s $N \times N/4$ band is thus the spectral counterpart of semi-coarsening,
 1201 and the honest classical comparison is the method a practitioner would use on this problem: *line*
 1202 Gauss–Seidel along x_2 (which contracts the x_1 -oscillatory modes by 0.35 – 0.45 per sweep, measured
 1203 on the modes $(N/2, 0)$ and $(N/4, 0)$ at 128^2 , where point Jacobi contracts them by 0.98) and line-
 1204 smoothed multigrid, which regains grid-independent convergence (asymptotically ≈ 0.04 per cycle
 1205 on one 128^2 instance, against ≈ 0.86 with point smoothing). Both are included, line GS as the sixth
 1206 pairing $\{\text{NO}, \text{line GS}\}$ and, together with point-smoothed multigrid and the CG variants, as classical
 1207 baselines timed in the same session **[pending]**. For the point-smoother pairings the router reaches h^2
 1208 in the time of one corrector call plus a few sweeps, as does the one-shot schedule, while HINTS with
 1209 $\tau = 25$, the reference of the first confirmatory run (revision 1), was 1.7 – $3.9\times$ slower there; in that
 1210 run the best fixed schedule, which is exactly one call at $t = 0$ followed by sweeps, was 3 – 7% faster
 1211 than the router at h^2 , the cost of the router’s per-step decision. At 10^{-8} the router was significantly
 1212 faster than the best fixed schedule in two of the five point-smoother pairings (Jacobi and damped
 1213 Jacobi) and at parity in the other three, within 4 – 7% of the cost-aware oracle; these numbers are
 1214 being regenerated under the revised protocol **[pending]**, and the five-seed trials for this equation are
 1215 **[pending]** (Table 21). The 512^2 grid is not run: the anisotropic corrector keeps full resolution along
 1216 x_1 ($N \times N/4$ sensors), so at 512^2 its least-squares branch would be a dense $65,536 \times 65,536$ matrix
 1217 (17 GB in single precision), which the current design cannot hold; a coarser x_1 resolution would
 1218 leave the point smoothers modes they cannot damp (Section E.7 discusses exactly this regime).

Table 16: Anisotropic diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 (64 instances, six pairings including line GS) and 256^2 (16 instances) grids for every policy, and for the classical baselines without corrector (conventions as in Table 4; the point-smoother-only runs are capped at 60,000 and 40,000 iterations and never reach h^2).

[results pending: confirmatory run in progress]

Table 17: Anisotropic diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$) and the best fixed period, with two-sided Wilcoxon p -values and bootstrap intervals; the classical baselines (point- and line-smoothed multigrid, CG variants) are timed in the same session.

[results pending: confirmatory run in progress]

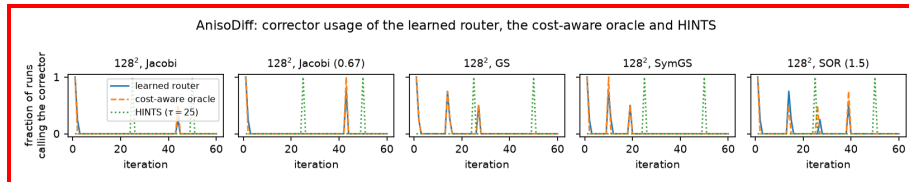


Figure 8: Anisotropic diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

Table 18: Anisotropic diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

[results pending: confirmatory run in progress]

Table 19: Anisotropic diffusion: measured per-iteration costs and macro-action sizes.

[results pending: confirmatory run in progress]

Table 20: Anisotropic diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

[results pending: confirmatory run in progress]

Table 21: Anisotropic diffusion, 128×128 : five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

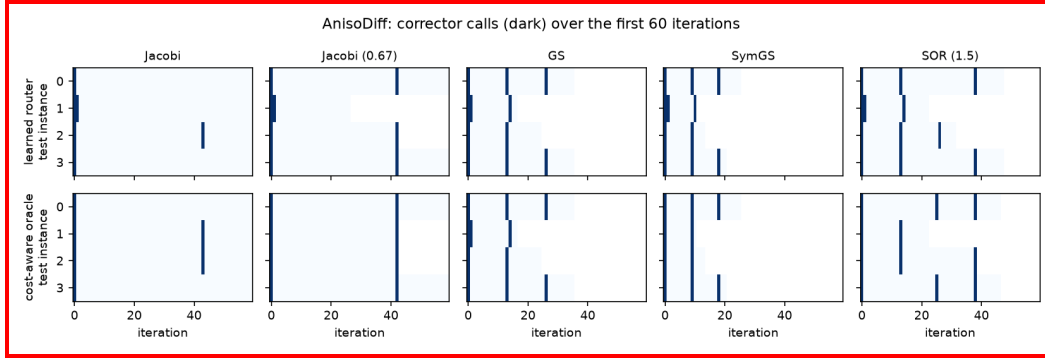


Figure 9: Anisotropic diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1219 E.5 Variable-coefficient diffusion

1220 The three equations above have constant coefficients on a periodic domain, so their discrete operators
1221 are diagonalised by the FFT: the exact band-limited inverse that the corrector approximates has a
1222 closed form, and a direct FFT solve is faster than any iterative method (Tables 4 and 10). To evaluate
1223 the hybrid where neither exists we add $-\nabla \cdot (a(\mathbf{x})\nabla u) = f$ on the same periodic domain with a
1224 fixed, smooth, positive coefficient field $a(\mathbf{x}) = \exp(\kappa z(\mathbf{x}))$, where z is a band-limited random field
1225 ($|k| \leq 4$, one fixed realisation, scaled by its maximum on a fixed 1024^2 reference grid, so that a is
1226 one grid-independent function sampled on every grid) and $\kappa = \frac{1}{2} \ln 10$, so that the prescribed contrast
1227 $\max a / \min a$ is 10 (attained up to the sampling of z , 9.99 on the grids used); the forcing distribution
1228 is unchanged. The discretisation is the conservative 5-point scheme with face-averaged coefficients
1229 (symmetric positive semidefinite, constants in its null space); reference solutions come from a sparse
1230 LU factorisation with one pinned unknown, which is also reported as a direct-solve baseline (its
1231 solve time with the factorisation cached, the factorisation itself costing ≈ 0.5 s at 128^2). Geometric
1232 multigrid rediscretises the coarse operators with block-averaged coefficients and contracts the error
1233 by ≈ 0.1 per V(2,2)-cycle. The corrector is the same least-squares DeepONet as before—a linear
1234 map on the 64×64 band—but fitted to (residual, error) pairs of *this* operator, whose band-limited
1235 inverse $R(\mathcal{L}_h)^{-1}P$ is not diagonal in the Fourier basis and has no closed form. It is nevertheless
1236 a fixed 4096×3969 matrix that could be assembled directly from 4096 sparse solves, one per
1237 sensor basis function; the least-squares fit on random pairs recovers it to the same $\approx 10^{-6}$ as on
1238 the constant-coefficient equations. What changes is that this matrix is no longer the inverse of the
1239 band-restricted error: a variable coefficient couples the band to the modes outside it, so an out-of-band
1240 residual leaves band error that the corrector does not see, and a band correction leaves out-of-band
1241 error behind. That coupling, not fit error, is why the corrector is called repeatedly along a solve and
1242 why the router’s decisions matter beyond the first call; what this experiment tests is routing on an
1243 operator without Fourier structure (no shared eigenbasis with the smoothers), not that the correction

1244 must be learned. The coefficient field is fixed rather than sampled, so this experiment tests a corrector
 1245 learned for one operator without an analytic inverse, not generalisation across coefficient fields; the
 1246 latter would require the coefficient-conditioned branch of the original DeepONet and is left to future
 1247 work. Tables 22 to 27 and fig. 10 report the same quantities as for the other equations on the 128^2
 1248 and 256^2 grids.

Table 22: Variable-coefficient diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 and 256^2 grids for every pairing and policy, and for the classical baselines without corrector, including the sparse direct solve (conventions as in Table 4).

[results pending: confirmatory run in progress]

Table 23: Variable-coefficient diffusion: paired median speedup [bootstrap 95% interval] of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=15$), the best fixed schedule, the one-shot schedule, multigrid alone and multigrid-preconditioned CG, with Holm-corrected two-sided Wilcoxon significance (conventions as in Table 5).

[results pending: confirmatory run in progress]

(figure pending)

Figure 10: Variable-coefficient diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=15$) for every pairing (columns) and grid (rows).

Table 24: Variable-coefficient diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

[results pending: confirmatory run in progress]

Table 25: Variable-coefficient diffusion: measured per-iteration costs and macro-action sizes.

[results pending: confirmatory run in progress]

Table 26: Variable-coefficient diffusion, 128×128 : iteration-based metrics (conventions as in Table 8).

[results pending: confirmatory run in progress]

Table 27: Variable-coefficient diffusion: five-seed router-training trials (conventions as in Table 9).

[results pending: confirmatory run in progress]

1249 E.6 Decision granularity

1250 The cost-aware router decides once per macro-action, and the macro-action of a cheap solver is as
 1251 expensive as one corrector call—32 Jacobi or 11 Gauss–Seidel sweeps at 128^2 with the compiled
 1252 kernels. Below truncation level the best fixed schedules at 10^{-8} are $\tau=15$ (first call at 0) (3), one-shot
 1253 (2), $\tau=10$ (first call at 0) (2), $\tau=5$ (first call at 0) (2), $\tau=25$ (first call at 0) (2), $\tau=10$ (1) (pairings
 1254 in parentheses), mostly phase-shifted HINTS with short periods, schedules the cost-aware oracle
 1255 cannot express because its decisions are too coarse, a plausible cause of the router’s losses in several
 1256 pairings at 10^{-8} (Tables 5, 11 and 17). The granularity is a free parameter of the construction of
 1257 Section 4: the unit of cost can be any fraction of a corrector call, actions dearer than the unit (here

the corrector itself) being compared per unit of cost. Table 28 repeats the 128^2 study with the unit set to one quarter of a corrector call (about a quarter of those sweeps per decision; the corrector is then the one action dearer than the unit and is scored per unit of cost), routers retrained with the same recipe, on the same confirmatory instances, for every pairing and for the nested two-member Jacobi ensemble.

Table 28: Decision granularity at 128×128 : median time to h^2 and 10^{-8} of the cost-aware oracle and the learned router with the unit of cost equal to one corrector call (main results) and to one quarter of it, and, in both cases, the router’s paired work-unit ratio at 10^{-8} against the main run’s best fixed schedule on the same instances (the ablation runs a reduced schedule family and reuses the main run’s fixed-schedule work units).

[results pending: confirmatory run in progress]

E.7 Solver ensembles

When can a second classical solver help? The cost-aware oracle is a per-decision comparison of macro-actions, so a router over $\text{NO} \cup \mathcal{W}$ can only beat the best pairwise router of its members if, along the way to the tolerance, the error passes through regimes in which *different* members give the largest reduction per unit cost. We screened this ceiling exhaustively before training any router: for every subset $\mathcal{W}$ of up to three members of {Jacobi, Jacobi (0.67), GS, SymGS, SOR (1.5), multigrid} (multigrid on the isotropic equations only), every equation and grid, we ran the cost-aware oracle over $\text{NO} \cup \mathcal{W}$ on the test instances and compared its work-unit time to that of the oracle over each member alone (Table 29). Two facts emerged in the development screening (**[provisional]**). At $\varepsilon = h^2$ no ensemble beat the best single solver anywhere: one corrector call followed by one or two macro-actions of the cheapest smoother already reaches truncation-level accuracy, so there is no second regime to exploit. At $\varepsilon = 10^{-8}$, after the corrector has removed the smooth band, the smoother has to damp the whole band $|k| > 31$, whose sub-bands favour different solvers: undamped Jacobi is the cheapest per unit cost on the modes near the band edge (it contracts them by $\cos(2\pi \cdot 32/N)$ -like factors that damped Jacobi cannot match), but it does not damp the near-Nyquist modes at all, which damped Jacobi and GS remove in a few sweeps. In the development screening (**[provisional]**: Table 29 is computed with the development-run macro-action sizes; the confirmatory screening with the compiled kernels’ sizes is pending) the oracle over {Jacobi, Jacobi (0.67)}—a *nonstationary* Jacobi iteration whose damping parameter is chosen per macro-action—was therefore faster than every single solver, by 13–28% on four of the six (equation, grid) settings at 128^2 and 256^2 , and adding a third member (GS) changed nothing, i.e., the gain saturates. Whether this ceiling survives the coarser macro-actions of the compiled kernels is exactly what the nested-ensemble runs below measure. At 512^2 (development screening) multigrid dominated every subset (its cycle removes all sub-bands at once), so the oracle over any ensemble containing it equalled the oracle over {NO, multigrid}: adding members neither helped nor hurt. The “difficulty” that matters for ensembles is thus not the equation but the width of the spectrum left to the classical members after the corrector, which grows with the grid (fixed band) and with the tolerance.

Nested ensembles. Table 30 evaluates the learned router over the nested sets {Jacobi}, {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} on every equation at 128^2 and 256^2 , and at 128^2 also over the four ensembles of Section 7 ({Jacobi, GS} up to all five solvers); the single-member routers are the pairwise routers of Sections E.2 to E.4, evaluated in the same session on the same instances, and all members keep the macro-action sizes of their pairwise runs. The table reports live and work-unit times to h^2 , 10^{-6} and 10^{-8} for the router and its oracle, and the paired speedup at 10^{-8} over the best single-member router (and of the ensemble oracle over the best single-member oracle) with its p -value; work units are the primary measure, since they are determined by the decision sequence alone. All cells of this table are **[pending]** under the revised protocol; the description below is of the first confirmatory run (revision 1, 39-sweep Jacobi macro-actions), and the macros will report the revised values. *Separation*: at $\varepsilon = 10^{-8}$ the ensemble router is significantly faster than every single-member router in **[pending]** of the **[pending]** ensemble cells, and the ensemble *oracle* is **[pending]**–**[pending]** of the best single-member oracle in work units (in the first run: no wins, and oracles within 6%, i.e., the greedy rule itself found no second regime to exploit). That differed from the development runs, in which the two-member Jacobi set was 8–15%

1305 faster than damped Jacobi alone on convection–diffusion and anisotropic diffusion (Table 33), and
 1306 the difference is plausibly the decision granularity. With the numpy sweeps a Jacobi macro-action
 1307 was about ten sweeps, and the oracle alternated undamped Jacobi—the cheapest member on the
 1308 band-edge modes $|k| \approx 32$, which it contracts by about 0.5 per sweep at 128^2 but which leaves the
 1309 Nyquist modes untouched—with damped Jacobi, which removes them. With the compiled sweeps a
 1310 damped-Jacobi macro-action is 32 sweeps: it contracts the band-edge modes by $0.665^{32} \approx 2 \times 10^{-6}$
 1311 and the Nyquist modes by $0.34^{32} \approx 1 \times 10^{-15}$ (single-mode factors at 128^2), which suggests that
 1312 the band the corrector leaves is removed within one or two macro-actions, leaving a nonstationary
 1313 schedule little to gain; the granularity ablation tests this hypothesis. Section E.6 repeats these cells
 1314 with a four times finer unit and tests directly whether the gain returns **[pending]**; the 256^2 grids,
 1315 where the band left to the smoothers is twice as wide and needs several macro-actions, are the other
 1316 place it could. *Saturation*: in the first run, adding GS to the Jacobi pair changed the oracle’s schedule
 1317 by at most a few percent, because GS never won a cost-aware comparison against the Jacobi pair.
 1318 *Monotonicity*: the ensemble routers’ ratios against the best single-member router in work units are
 1319 **[pending]**–**[pending]** at h^2 and **[pending]**–**[pending]** at 10^{-8} , with **[pending]** of the **[pending]**
 1320 cells more than 5% slower (in the first run these were sets containing both GS and SymGS, where
 1321 the router’s imitation of its oracle degraded among near-tied members while the oracles themselves
 1322 stayed within 6% of the best single oracle). A greedy rule is not monotone in its action set in general,
 1323 and its learned imitation is less so; the table reports how far the losses go.

Table 29: Oracle-level screening of ensembles (**[provisional: development-run costs and instances]**): for every equation and grid, the fastest single solver and the fastest ensemble of up to three members under the cost-aware oracle (median work-unit time over 16 test instances), at $\varepsilon = h^2$ and 10^{-8} ; the ratio is the speedup of the ensemble over the best single solver, and the last column is the ratio for {Jacobi, Jacobi (0.67)}.

[results pending: confirmatory run in progress]

Table 30: Nested ensembles: median time to $\varepsilon \in \{h^2, 10^{-6}, 10^{-8}\}$ of the learned router (live and work units) and of the cost-aware oracle (work units) over $\text{NO} \cup \mathcal{W}$, for the single members and the nested two- and three-member sets, all evaluated in one session on the same instances (64 at 128^2 , 32 at 256^2 , 16 for anisotropic diffusion at 256^2). The last columns give the paired speedup at 10^{-8} over the best single-member router (resp. oracle) with the two-sided Wilcoxon p -value; bold: $\geq 1.1\times$ and significant after a Holm correction over all tests of the table.

[results pending: confirmatory run in progress]

1324 E.8 Overheads and amortisation

1325 Table 31 lists the measured cost of every operation as a function of N , including one decision of the
 1326 LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one LSTM
 1327 decision costs **[pending]** ms, about **[pending]** Jacobi sweeps, which is why the wall-clock study uses
 1328 the scalar-feature router ($\approx 5 \mu\text{s}$, independent of N). In situ, inside the timed replays, the router’s
 1329 decisions cost 16–20 μs each (the first decision of a solve and every decision after a macro-action
 1330 find cold caches; the 5 μs figure is a warm-loop median), i.e. 2.6–3.3% of the router’s time to h^2 (2–2
 1331 decisions) and 0.9–2.2% of its time to 10^{-8} , as instrumented directly inside the replays. Separately,
 1332 the paired difference between the router’s and the cost-aware oracle’s time at h^2 is 22–59 μs per
 1333 solve; the two run as separate replays with decisions that agree on **[pending]**–**[pending]**% of the
 1334 instances, so this gap is consistent with, but not a measurement of, the decision cost (an earlier version
 1335 of this appendix put the decisions at “well under 1%” on the basis of the warm-loop figure). The
 1336 corrector calls account for an estimated 60–90% of the router’s time to h^2 (number of calls times
 1337 the calibrated per-call cost; per-cell medians over the 128^2 pairwise cells). Two further accounting
 1338 checks: the charged time of a router solve is 99–99% of the outer elapsed time of its replay, the
 1339 remainder being the Python driver’s loop and timer overhead, which every method pays per iteration;
 1340 and the router’s live time to h^2 is 0.99–1.22 times its work-unit time, the per-operation costs being

measured in the same form as the replay charges them. Table 32 accounts for training: the corrector (shared with HINTS) is fit in minutes, the router in seconds to a few minutes per pairing, and the break-even number of solves after which the router’s own training cost is recovered relative to HINTS ($\tau=15$) with the same corrector is [pending]–[pending] solves for the GS pairing at each equation and grid (Table 32; [pending] of those cells have no positive saving and [pending] have no recorded router-training time).

Table 31: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

[results pending: confirmatory run in progress]

Table 32: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and DAGger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS ($\tau=15$) and to the solver alone, and the number of solves after which the router’s training cost is recovered relative to HINTS.

[results pending: confirmatory run in progress]

E.9 Development versus confirmatory runs

Several choices of this study were made after looking at test results: the corrector band and the anisotropic sensor grid, the router of the convection–diffusion Jacobi pairing at 512^2 (retrained with more rollouts after the first router missed h^2 on 4 of 16 development instances, Section E.1), and the imitation budget of the ensemble routers (raised after the pairwise recipe imitated its oracle imperfectly on one development cell, Section E.7); the optimistic best-schedule baseline is tuned on the test set by design and labelled as such. All of these were made on the development set (seed 72), with the earlier numpy/scipy sweeps, the earlier forcing sampler and $\tau = 25$ as the HINTS reference (Section E.1), which is why the table compares against $\tau = 25$ in both runs. Every result in this appendix comes from a single evaluation of the frozen configurations on the confirmatory set (seed 73) with the compiled kernels and three timed replays. Table 33 lists, for every cell present in both, the development and the confirmatory speedups: the two differ both through the instances and through the kernels—the compiled sweeps are $4\text{--}5\times$ faster, so the corrector is worth $2\text{--}3\times$ more sweeps than in the development runs and the margins over fixed schedules at h^2 shrink accordingly—and the table shows exactly how much of the development-phase margin survives.

Table 33: Development (seed 72, numpy/scipy sweeps, one timed replay) versus confirmatory (seed 73, compiled kernels, three timed replays) paired median speedups of the learned router, for every pairwise cell at $\varepsilon = h^2$ against HINTS ($\tau=25$, the reference of the development runs) and against the best fixed schedule available in that run, and for every nested ensemble at 10^{-8} in work units against the best single-member router and against its oracle; two-sided Wilcoxon p -values in parentheses, censoring classes as in the speed tables; bold: ratio $\geq 1.1\times$ and significant after a Holm correction over the table.

Equation	N	Pairing / ensemble	development (seed 72, numpy kernels)		confirmatory (seed 73, compiled kernels)	
			vs. HINTS-25 or best single	vs. best schedule or oracle	vs. HINTS-25 or best single	vs. best schedule or oracle
Poisson	128^2	{NO, Jacobi}	$2.68\times$ ($< 10^{-10}$)	$1.21\times$ (4×10^{-8})	$1.62\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 0.095)
	128^2	{NO, Jacobi (0.67)}	$2.90\times$ ($< 10^{-10}$)	$1.28\times$ (4×10^{-9})	$1.57\times$ ($< 10^{-10}$)	$0.98\times$ (slower, 0.158)
	128^2	{NO, GS}	$6.75\times$ ($< 10^{-10}$)	$1.94\times$ ($< 10^{-10}$)	$2.89\times$ ($< 10^{-10}$)	$0.95\times$ (slower, 9×10^{-10})
	128^2	{NO, SymGS}	$9.21\times$ ($< 10^{-10}$)	$2.34\times$ ($< 10^{-10}$)	$3.73\times$ ($< 10^{-10}$)	$0.95\times$ (slower, $< 10^{-10}$)
	128^2	{NO, SOR (1.5)}	$10.3\times$ ($< 10^{-10}$)	$3.22\times$ ($< 10^{-10}$)	$4.14\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 10^{-6})
	128^2	{NO, Multigrid}	$1.96\times$ ($< 10^{-10}$)	$1.88\times$ ($< 10^{-10}$)	$0.99\times$ (slower, 0.042)	$0.97\times$ (slower, $< 10^{-10}$)
Convection–diffusion	128^2	{NO, Jacobi}	$2.95\times$ ($< 10^{-10}$)	$1.34\times$ (10^{-10})	$1.61\times$ ($< 10^{-10}$)	$0.95\times$ (slower, 3×10^{-6})
	128^2	{NO, Jacobi (0.67)}	$3.14\times$ ($< 10^{-10}$)	$1.32\times$ (10^{-9})	$1.53\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 4×10^{-4})
	128^2	{NO, GS}	$6.76\times$ ($< 10^{-10}$)	$2.60\times$ ($< 10^{-10}$)	$2.93\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 0.048)
	128^2	{NO, SymGS}	$8.70\times$ ($< 10^{-10}$)	$2.40\times$ ($< 10^{-10}$)	$3.69\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 10^{-7})
	128^2	{NO, SOR (1.5)}	$8.89\times$ ($< 10^{-10}$)	$2.40\times$ ($< 10^{-10}$)	$4.73\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 10^{-5})
	128^2	{NO, Multigrid}	$2.07\times$ ($< 10^{-10}$)	$2.01\times$ ($< 10^{-10}$)	$0.96\times$ (slower, 0.084)	$0.96\times$ (slower, 6×10^{-10})

E.10 Verification of the theoretical assumptions

Theorem 4.1 and Propositions 4.2–4.3 assume that every error propagation map $G_j = I - C_j \circ \mathcal{L}_h^a$ is Lipschitz with constant $\rho_j \leq 1$ (with at least one strict contraction in the optimal sequence, respectively a shared unitary eigenbasis), zero-preserving, and—for postfix monotonicity—simultaneously diagonalisable by a unitary matrix; Theorem 5.1 assumes that at every state at least one solver leaves a positive fraction of the error. Table 34 checks these conditions numerically for every operation of the study—finite diagnostics (a power-iteration estimate of the Lipschitz constant, commutators on random vectors, exhaustive search over short horizons), none of which is a certificate over the whole input space—on the mean-free subspace (the constant mode is the null space of the periodic operator and is excluded from the data and from every iteration): the Euclidean Lipschitz constant $\|G_j\|_2$ (power iteration on $G_j^\top G_j$; exact from the Fourier symbol for Jacobi), the Lipschitz constant $\|G_j\|_A = \|A_s^{1/2} G_j A_s^{-1/2}\|_2$ in the energy norm of the symmetric part A_s of the discrete operator, the same for the macro-action $G_j^{m_j}$, the spectral radius, the smallest observed contraction $\sigma_{\min} = \min \|G_j x\|/\|x\|$, $\|C_j(0)\|$, and the commutator $\|G_j G_{\text{NO}} - G_{\text{NO}} G_j\|$ on random vectors. Table 36 evaluates the quantities of Theorem 4.1 and Theorem 5.1 along the cost-aware oracle’s own path to $\varepsilon = 10^{-8}$ (a horizon of tens of macro-actions; to h^2 the path has one or two steps).

What holds, and in which norm. (i) Every classical solver is zero-preserving exactly and contracts in the sense that matters for convergence: the spectral radius of G_j is below 1 in every setting (at most **[pending]**), and so is the energy-norm Lipschitz constant of the solver and of its macro-action (at most **[pending]**). The *Euclidean* Lipschitz constant, however, exceeds 1 for Gauss–Seidel, SOR and symmetric GS (up to **[pending]** for SOR), whose iteration matrices are non-normal; the proof of Proposition 4.2 uses only the triangle inequality and the composition rule for Lipschitz constants, so it holds verbatim with $\|\cdot\|_2$ replaced by the energy norm, in which the condition is satisfied by all classical solvers in all settings—but that statement concerns the greedy rule and objective defined with the energy norm, whereas the oracle and the router of this study minimise the Euclidean error. For the evaluated Euclidean rule, the strict-contraction premise of Proposition 4.2 therefore holds for the Jacobi-type solvers and fails for the non-normal sweeps, whose Euclidean expansion is confined to a single sweep (their spectral radii and macro-action contraction show the iteration is convergent). For (damped) Jacobi both constants agree (the maps are Fourier-diagonal); the values are close to 1 because the slowest modes (the smoothest ones, and the near-Nyquist mode for undamped Jacobi) are barely damped by a point smoother—which is exactly why the corrector is needed. (ii) The corrector’s map is zero-preserving, contracts every band mode to **[pending]** or better (on average to 10^{-4} – 10^{-6}) and leaves the modes outside the band untouched, so its Lipschitz constant is exactly 1 (not < 1): Theorem 4.2 in its strict form does not apply to the corrector, but Theorem 4.3 does, and its second condition—a shared unitary eigenbasis—holds exactly for the exact band-limited inverse together with (damped) Jacobi, all of which are diagonal in the Fourier basis, and holds for the learned corrector up to its fit error (measured commutators $\approx 10^{-6}$, at the corrector’s single-precision accuracy; replacing the learned corrector by the exact inverse does not change the oracle’s decisions, Section E.1), so for the ensembles $\{\text{NO}, \text{Jacobi}\}$, $\{\text{NO}, \text{Jacobi} (0.67)\}$ and $\{\text{NO}, \text{Jacobi}, \text{Jacobi} (0.67)\}$ with the exact band-limited inverse the objective is supermodular and postfix monotone and Theorem 4.1 applies with $\alpha = \mu = 1$; for the fitted corrector we measure the fit error and the commutators but do not propagate them through a perturbation bound (a small commutator is a diagnostic, not a certificate), so no guarantee is stated for it, and the exhaustive short-horizon check below is the evidence that the fitted maps behave like the exact ones. For GS, SOR, symmetric GS and multigrid the commutators are small but nonzero (10^{-3} – 10^{-1}) and no guarantee is claimed at any horizon. (iii) Invertibility, which an earlier version of the theory used for postfix monotonicity, is neither needed for the corrected result nor sufficient in general (Section A.3); for the record, a Gauss–Seidel or symmetric-GS sweep annihilates an error concentrated at the first grid point ($\sigma_{\min} = 0$), SOR leaves $(1 - \omega)$ of it ($\sigma_{\min} = 0.72$ – 0.83 in Table 34), undamped Jacobi annihilates the mode $(N/4, N/4)$, and the corrector annihilates the band. Postfix monotonicity ($\mu = 1$) is therefore guaranteed only for the Jacobi-type ensembles with the exact band-limited inverse, and is measured for the others. (iv) Table 35 checks Theorem 4.1 by exhaustive enumeration (in floating point, with errors clipped at a relative floor of 10^{-11}) for short horizons on the evaluated ensembles, in the Euclidean objective the oracle optimises and for the deployed cost-aware rule: for every pairing (including line GS on anisotropic diffusion), for $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$, for the two non-commuting sweeps $\{\text{GS}, \text{SOR}\}$, for $\{\text{GS}, \text{multigrid}\}$ and for all point smoothers together, the optimal sequence

1418 O of T macro-actions ($T = 4$, or 3 for $K = 6$) is found by exhaustive search on 8 test instances (4 for
 1419 $K = 6$), and μ (over the greedy prefixes), $\alpha(O)$ (over the greedy prefixes, and over every sequence
 1420 of length below T) and the bound itself are computed. Over all [pending] instance–ensemble pairs
 1421 the largest μ is [pending], the largest $\alpha(O)$ over the greedy prefixes is [pending] and over all
 1422 short prefixes [pending]; the numerical bound holds on [pending] pairs, and the on-path premises
 1423 that the recurrence proof of Theorem 4.1 uses—prefix monotonicity and the weak-supermodularity
 1424 inequality at the reported α on the greedy prefixes (not the global definition), the deployed rule
 1425 equal to Algorithm 1, and an exhaustive optimum resolved above the round-off floor—are verified,
 1426 as a finite numerical check, on [pending] pairs (on [pending] the optimum is at the floor and the
 1427 instance is numerically unresolved rather than certified; the deployed rule differs from Algorithm 1
 1428 on [pending], and the clipping would have changed the deployed path on [pending]); [pending]
 1429 greedy prefixes and [pending] prefixes overall show an expansion of the error by a single action of O ,
 1430 i.e. a failure of prefix monotonicity in the Euclidean norm, and [pending] greedy prefixes ([pending]
 1431 prefixes overall) violate the inequality at the reported α (the largest candidate α over greedy prefixes,
 1432 [pending], includes the pairs that fail). The greedy sequence’s error is within [pending] $\times$ of the
 1433 exhaustive optimum on every pair (the table gives medians and the worst cases per ensemble). The
 1434 check supports the optimality of the greedy rule over a few macro-actions; the horizons at which
 1435 the oracle runs to 10^{-8} are 10–400 macro-actions (Table 36), where no constant is established for
 1436 the non-normal sweeps: Proposition 4.2 would need contraction in the Euclidean norm, which they
 1437 lack (Table 34), and the worst-case constant in Table 36 is computed with energy-norm constants for
 1438 orientation only. The worst-case $\alpha(O)$ of Theorem 4.2, which only uses the Lipschitz constants, is
 1439 loose in comparison: along the oracle’s path to 10^{-8} (Table 36) $\sum_i \rho_{O_i}^2$ is close to T because each
 1440 ρ is dominated by the slowest mode, so the bound ranges from [pending] (the multigrid pairing,
 1441 whose cycle contracts everything) to 10^2 – 10^4 for the point smoothers, and is infinite on the few
 1442 instances where $\sum_i \rho_{O_i}^2 \geq T$. (The last two columns of Table 36 evaluate the ratio of Equation (7)
 1443 with the oracle’s own suffix in place of the unknown long-horizon O ; this is only a diagnostic, and its
 1444 maximum is attained trivially at the last prefix.) (v) Theorem 5.1 is stated for the per-state normalised
 1445 weights $\tilde{c}_j / \sum_k \tilde{c}_k$ that the router is trained with; its one hypothesis is that at every visited state at
 1446 least one solver leaves a positive fraction of the current error, i.e. $\max_j \tilde{c}_j / \|e^{(t)}\|^2 > 0$ (last column
 1447 of Table 36; the column before it shows that the per-step errors never exceed the current error), so the
 1448 surrogate’s Bayes consistency applies at every visited state.

Table 34: Numerical check of the assumptions of Propositions 4.2–4.3 for every operation of the
 study (mean-free subspace). **[Provisional: development run; the confirmatory run regenerates
 this table with the compiled kernels’ macro-action sizes.]** $\|\cdot\|_2$: Euclidean Lipschitz constant
 of the error propagation map (power-iteration estimate; exact from the symbol for Jacobi); $\|\cdot\|_A$:
 Lipschitz constant in the energy norm of the symmetric part of $\mathcal{L}_h$, for one iteration and for the
 macro-action of m_j iterations; ρ : spectral radius; $\sigma_{\min}$: smallest observed contraction (a unit error
 at the first grid point for the triangular sweeps, the minimum over Fourier modes otherwise; for the
 corrector, the largest remaining fraction of a band mode); $\|C_j(0)\|$: zero preservation; last column:
 commutator with the corrector’s map on random unit vectors (Proposition 4.3). For (damped) Jacobi
 the constants are exact from the Fourier symbol (the value with the Nyquist modes included; without
 them undamped Jacobi has $\|G\|_2 = \rho = 0.9994$ at 128^2).

[results pending: confirmatory run in progress]

Table 35: Exhaustive short-horizon check of Theorem 4.1 (128×128 , n test instances per row, T macro-actions, K actions including the corrector, deployed cost-aware rule): the optimal sequence O is found by exhaustive search; $\mu = \max_{t < T} g(S^t \oplus O)/g(O)$ over the greedy prefixes; $\alpha(O)$ over the greedy prefixes and over all sequences of length below T ; whether $g(S^T) \leq \mu(1 - \phi_T)g(O) + \phi_T g(\emptyset)$ holds; the ratio $g(S^T)/g(O)$ of the greedy to the optimal error (median and maximum over instances); the number of instances on which the greedy sequence is the optimum; and the numbers of instances on which the exhaustive optimum is at the round-off floor (numerically unresolved), on which the deployed rule coincides with Algorithm 1, of greedy prefixes and of all prefixes at which the weak-supermodularity inequality fails at the reported α , and of instances on which the on-path premises of the proof were verified. Errors below 10^{-11} relative are clipped to that floor.

[results pending: confirmatory run in progress]

Table 36: Quantities of Theorem 4.1, Proposition 4.2 and Theorem 5.1 along the cost-aware oracle’s path to $\varepsilon = 10^{-8}$ (8 test instances; medians unless stated): number of macro-actions T , $\sum_i \rho_{O_i}^2$ with the energy-norm constants of the chosen macro-actions, the per-instance margin $T - \sum_i \rho_{O_i}^2$ (median) and the resulting per-instance $\alpha(O)$ of Proposition 4.2 (median and range over the instances on which the margin is positive; ∞ otherwise), the empirical supermodularity ratio $\hat{\alpha}$ of Equation (7) evaluated with S the oracle’s prefix and S' its suffix (maximum and median over prefixes and instances), and the normalised bounds of Theorem 5.1: the largest ratio $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ over visited states ($\bar{E}$) and the smallest value of $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ ($E_{\min}$).

[results pending: confirmatory run in progress]

1449 F Limitations

1450 A primary limitation of our approach is the higher on-time computational cost requires to learn the
 1451 greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead.
 1452 While this cost is amortized during inference time and leads to improved solution quality, it may be
 1453 prohibitive in settings with limited computational resources or when rapid deployment is required.
 1454 On the deployment side, Section E shows what survives cost accounting: with a corrector that covers
 1455 the smooth half of the spectrum and a classical smoother that covers the rest, the router makes
 1456 one corrector call on 80–100% of the instances at truncation level and executes the same decision
 1457 sequence as the best static schedule on **[pending]–[pending]**% of them, without a search; below
 1458 truncation level the comparison with the best fixed schedule is pairing-dependent, and the paired
 1459 ratio of multigrid alone to the router is $0.03 \times - 0.92 \times$ at 128^2 . The corrector we use is specific to
 1460 the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost
 1461 grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below
 1462 truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is
 1463 restricted to single-thread CPU execution; with a corrector band that does not grow with the grid, the
 1464 gains over fixed schedules shrink at 512^2 for the point smoothers, whereas pairing the corrector with
 1465 geometric multigrid is expected to restore them (Section E; **[512² pending]**). Ensembles of classical
 1466 solvers can pay off only where the error passes through regimes that favour different members; in the
 1467 development screening a single well-chosen smoother sufficed at truncation level (Section E.7). For
 1468 the constant-coefficient periodic test problems the discrete operator is diagonalised by the FFT, and
 1469 the resulting direct solve, reported in every table, took $1/3.34 \times - 1/2.98 \times$ of the best router’s time at
 1470 128^2 ; the hybrid is meant for problems on which no such direct diagonalisation exists—where the
 1471 band-limited inverse has to be assembled or fitted rather than read off a symbol, and the corrector is
 1472 then inexact—which is why Section E also reports a variable-coefficient diffusion problem.

1473 Additionally, the effectiveness of the learned router depends on the neural operator exhibiting
 1474 heterogeneous performance across samples. When the neural operators produces predictions of
 1475 relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a
 1476 fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity

1477 can be sensitive to training choices, and neural operators themselves can be finicky to train. In the
1478 wall-clock study the number of corrector calls to truncation level is one on 80–100% of the instances
1479 (Section 7); whatever heterogeneity adaptive routing exploits there lies below truncation level, in
1480 when the corrector is worth calling again.

1481 G LLM Usage

1482 LLMs were used substantially in preparing this paper, and we disclose this precisely. The text of
1483 the original submission was drafted by the authors, with ChatGPT and Gemini used for feedback
1484 on grammar and clarity. The revision that responds to the reviews was produced with Claude Code
1485 (Anthropic), operated and directed by the authors: the wall-clock benchmark pipeline (the compiled
1486 stencil kernels, the cost-aware macro-action environment, the scalar-feature router and its training
1487 loop, the corrector fitting, the baseline, table and figure generators, and the assumption and theorem
1488 checkers) was written largely by the assistant under the authors’ direction and was reviewed, tested
1489 and run by the authors; the corrections to the theory in this revision (the μ -slack in Theorem 4.1, the
1490 corrected constant and hypotheses of Proposition 4.2, the counterexample to postfix monotonicity,
1491 the unitary-eigenbasis condition, and the restatement of Theorem 5.1 for normalised weights) were
1492 drafted with the assistant’s help in response to reviewer findings and were verified by the authors;
1493 and much of the new text of Section 7 and Section E was drafted by the assistant from the generated
1494 tables and edited by the authors. The theorems and proof strategies of the original submission predate
1495 the use of the assistant. The research questions, the framing of the method, and the decisions about
1496 what to run and what to claim are the authors’. Independent reviews of the manuscript and code were
1497 also obtained from LLM-based reviewers; their findings are addressed in this revision.

1498 NeurIPS Paper Checklist

1499 1. Claims

1500 Question: Do the main claims made in the abstract and introduction accurately reflect the
1501 paper’s contributions and scope?

1502 Answer: [Yes]

1503 Justification: The abstract and introduction state the two-regime empirical outcome (parity
1504 with the best static schedule at truncation level; mixed results against the best fixed schedule
1505 below it; multigrid alone faster below truncation level) with counts and ranges computed
1506 from the tables, and the theoretical claims with their hypotheses (unitary eigenbasis for
1507 postfix monotonicity, measured slack otherwise; per-decision consistency for the full-
1508 information router). At the time of this draft several confirmatory cells are still being
1509 regenerated and are marked as pending in the text.

1510 Guidelines:

- 1511 • The answer [N/A] means that the abstract and introduction do not include the claims
1512 made in the paper.
- 1513 • The abstract and/or introduction should clearly state the claims made, including the
1514 contributions made in the paper and important assumptions and limitations. A [No] or
1515 [N/A] answer to this question will not be perceived well by the reviewers.
- 1516 • The claims made should match theoretical and experimental results, and reflect how
1517 much the results can be expected to generalize to other settings.
- 1518 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1519 are not attained by the paper.

1520 2. Limitations

1521 Question: Does the paper discuss the limitations of the work performed by the authors?

1522 Answer: [Yes]

1523 Justification: The paper discusses its limitations in the Limitations section (Appendix F),
1524 including the constant-coefficient periodic scope of the corrector, the single-thread CPU
1525 setting, the test-set-selected optimistic baselines and the pairing-dependent results below
1526 truncation level.

1527 Guidelines:

- 1528 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1529 that the paper has limitations, but those are not discussed in the paper.
- 1530 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1531 • The paper should point out any strong assumptions and how robust the results are to
1532 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1533 model well-specification, asymptotic approximations only holding locally). The authors
1534 should reflect on how these assumptions might be violated in practice and what the
1535 implications would be.
- 1536 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1537 only tested on a few datasets or with a few runs. In general, empirical results often
1538 depend on implicit assumptions, which should be articulated.
- 1539 • The authors should reflect on the factors that influence the performance of the approach.
1540 For example, a facial recognition algorithm may perform poorly when image resolution
1541 is low or images are taken in low lighting. Or a speech-to-text system might not be
1542 used reliably to provide closed captions for online lectures because it fails to handle
1543 technical jargon.
- 1544 • The authors should discuss the computational efficiency of the proposed algorithms
1545 and how they scale with dataset size.
- 1546 • If applicable, the authors should discuss possible limitations of their approach to
1547 address problems of privacy and fairness.
- 1548 • While the authors might fear that complete honesty about limitations might be used by
1549 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1550 limitations that aren’t acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: All theoretical results include clearly stated assumptions in the statement of theorems. Complete proofs are provided in Appendices A and B (greedy guarantee) and C (surrogate consistency) with cross-references from the main text.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including the protocol, training procedures and implementation details described in Section 7 and Appendix E.1, and the code, run scripts and README that produce every table and figure from scratch.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies all training and evaluation details in Section 7 and Appendix E.1 (protocol, corrector fitting, router training, baselines, statistics); Appendix C documents the pipeline of the original submission whose experiment was withdrawn.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Every speedup in the speed tables and in the main-text table is a paired median ratio over test instances with a bootstrap 95% interval and a two-sided Wilcoxon signed-rank test, Holm-corrected within each table (Appendix E.1); the nested-ensemble, granularity and development-versus-confirmatory tables use the same tests and Holm correction without intervals; the AUC tables report means with standard errors and one-sided paired t-tests on log AUC; the seed trials report means and standard deviations over five router-training seeds and count seeds significant in the router's direction at $p < 0.01$.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper provides details on compute resources (single-thread CPU timings on one machine, training times, amortisation) in Appendix E.1 and E.8.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This paper does not involve any human subjects and all data has been simulated. This research focuses on numerical methods for physical systems and does not raise concerns related to safety, security, discrimination, surveillance, deception & harassment, environment, human rights, and bias & fairness.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses broader impacts in the introduction section. No direct negative societal impacts are anticipated.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release any models or datasets at all. It focuses on methodological contributions for numerical solvers using simulated data. Therefore, safeguards are not applicable.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- 1761 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1762 should describe how they avoided releasing unsafe images.
1763 • We recognize that providing effective safeguards is challenging, and many papers do
1764 not require this, but we encourage authors to take this into account and make a best
1765 faith effort.

12. Licenses for existing assets

1767 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1768 the paper, properly credited and are the license and terms of use explicitly mentioned and
1769 properly respected?

1770 Answer: [N/A]

1771 Justification: The paper does not use any external datasets or pretrained models. All
1772 experiments are conducted using simulated data and models trained in-house.

1773 Guidelines:

- 1774 • The answer [N/A] means that the paper does not use existing assets.
1775 • The authors should cite the original paper that produced the code package or dataset.
1776 • The authors should state which version of the asset is used and, if possible, include a
1777 URL.
1778 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
1779 • For scraped data from a particular source (e.g., website), the copyright and terms of
1780 service of that source should be provided.
1781 • If assets are released, the license, copyright information, and terms of use in the
1782 package should be provided. For popular datasets, paperswithcode.com/datasets
1783 has curated licenses for some datasets. Their licensing guide can help determine the
1784 license of a dataset.
1785 • For existing datasets that are re-packaged, both the original license and the license of
1786 the derived asset (if it has changed) should be provided.
1787 • If this information is not available online, the authors are encouraged to reach out to
1788 the asset's creators.

13. New assets

1790 Question: Are new assets introduced in the paper well documented and is the documentation
1791 provided alongside the assets?

1792 Answer: [Yes]

1793 Justification: We release code for our method which is documented with a detailed
1794 README.

1795 Guidelines:

- 1796 • The answer [N/A] means that the paper does not release new assets.
1797 • Researchers should communicate the details of the dataset/code/model as part of their
1798 submissions via structured templates. This includes details about training, license,
1799 limitations, etc.
1800 • The paper should discuss whether and how consent was obtained from people whose
1801 asset is used.
1802 • At submission time, remember to anonymize your assets (if applicable). You can either
1803 create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

1805 Question: For crowdsourcing experiments and research with human subjects, does the paper
1806 include the full text of instructions given to participants and screenshots, if applicable, as
1807 well as details about compensation (if any)?

1808 Answer: [N/A]

1809 Justification: The paper does not involve crowdsourcing or any research with human subjects.
1810 All experiments are conducted using simulated data.

1811 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve human subjects. All experiments are conducted using simulated data. Therefore, IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are not a component of the method, but an LLM coding assistant was used substantially in this revision to implement the benchmark pipeline, to help draft the corrections to the theory and to draft text from the generated tables, under the authors' direction and with the authors' verification; the LLM Usage section (Appendix G) describes this precisely.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.